PythonEX — Full Course

UNIT 1 45 min

Your First Python Program

Print output, store values in variables, write comments, and read error messages.

By the end of this unit you will be able to

- Run a Python statement and explain what `print()` does
- Create variables with the assignment operator `=`
- Follow the naming rules for variables and choose readable names
- Write comments that explain intent
- Read a simple error message and find the line that caused it

Why Python

Python reads close to plain English, which makes it a good first language and a strong tool for working professionals. The same language runs web sites, data analysis, automation, and machine-learning models. Everything you learn in this course applies to all of those.

Your first program

A Python program is a list of **statements**. Python runs them from top to bottom, one at a time. The `print()` function displays a value on the screen (the output).

hello.py

```
print("Hello, world!")
print("Python is running.")
```

> **Tip**
> Press **Run** ▶ on any code box to execute it in your browser, or copy the code and paste it into the Sandbox page.

Variables store values

A **variable** is a name that points to a value. You create one with the assignment operator `=`. The right side is evaluated first, then the result is stored under the name on the left. Read `=` as "gets", not as "equals".

variables.py

```
name = "Ada"
age = 36
print(name)
print(age)
```

Trace: a variable is reassigned TRACE

```
1  x = 5
2  y = x + 2
3  x = 10
4  print(y)
```

Step	Line run	Variables after the line	Output so far
1	1	`x = 5`	`—`
2	2	`x = 5, y = 7`	`—`
3	3	`x = 10, y = 7`	`—`
4	4	`x = 10, y = 7`	`7`

Naming rules

- Names may use letters, digits, and underscores `_`.
- A name cannot start with a digit. `2nd_place` is illegal; `second_place` is fine.
- Names are **case-sensitive**. `Score` and `score` are two different variables.
- Names cannot be Python keywords such as `for`, `if`, `while`, or `True`.
- Use **snake_case** (lowercase words joined by underscores). It is the Python convention.

Name	Legal?	Why
`total_price`	Yes	Descriptive snake_case
`x1`	Yes	Letters then a digit is fine
`1st`	No	Starts with a digit
`my-name`	No	A hyphen is read as minus
`class`	No	Reserved keyword

Comments

A comment starts with `#`. Python ignores everything after it on that line. Use comments to explain *why* code does something, not to repeat what the code already says.

commented.py

```
# Store the sales tax rate so it is easy to change later.
TAX_RATE = 0.07

price = 40          # the price before tax
print(price * (1 + TAX_RATE))
```

Reading errors

When Python cannot run a line, it stops and prints an error. The last line says what went wrong, and the line above it says where. Errors are information, not failure.

broken.py

```
print(score)
```

That produces `NameError: name 'score' is not defined`. The name `score` was never assigned. Fix it by creating the variable first.

Predict the output THINK FIRST

Before you run anything, decide what each program prints.

```
print(2 + 3)
```

Answer: `5`

```
print("2 + 3")
```

Answer: `2 + 3`
Inside quotes, Python treats the text literally. It is a string, not math.

```
a = 1
b = a
a = 2
print(b)
```

Answer: `1`
`b` received the value of `a` at that moment. Later changes to `a` do not affect `b`.

```
print(score)
```

Answer: `NameError`
The name was never assigned, so Python cannot find it.

Practice

▼ **Exercise 1 — Introduce yourself**

Create three variables: `name` (text), `favorite_number` (a whole number), and `city` (text). Print each one on its own line.

▼ **Hint**

Text goes in quotes. Numbers do not.

▼ **Exercise 2 — Swap two values**

Given `a = 1` and `b = 2`, write code that swaps them so `a` ends with 2 and `b` ends with 1. Use a third variable called `temp`.

▼ **Hint**

Save one value before you overwrite it.

Discussion

1. Why do you think Python uses the `=` symbol for assignment when math uses it for equality? What does that cost a beginner?
2. Give an example of a comment that repeats the code and one that explains why. Which one is more useful six months from now?
3. Which naming rule do you think causes the most confusion for new programmers, and why?

Key terms

Statement

One complete instruction the interpreter runs, such as `print("hi")` or `x = 3`.

Interpreter

The program that reads Python code and runs it. Python code runs line by line through the interpreter.

Variable

A name that refers to a value stored in memory. The value can be changed by assigning a new one.

Assignment

Storing a value under a name using `=`. The right side is computed first.

print()

A built-in function that writes its arguments to the output, separated by spaces, followed by a newline.

Comment

Text after `#` that Python ignores. Used to explain intent to human readers.

Keyword

A word reserved by the language (for example `if`, `for`, `True`). It cannot be used as a variable name.

snake_case

Naming style with lowercase words joined by underscores, such as `total_price`. Python's convention for variables and functions.

NameError

The error raised when code uses a name that has not been assigned.

SyntaxError

The error raised when code breaks the grammar of the language, such as a missing quotation mark or parenthesis.

See all terms in the glossary →

UNIT 2 60 min

Data Types and Strings

Integers, floats, strings, and booleans; checking types; converting between them; working with text.

By the end of this unit you will be able to

- Identify the common types: `int`, `float`, `str`, `bool`
- Check a value's type with `type()` and convert with `int()`, `float()`, `str()`
- Explain why `0.1 + 0.2` is not exactly `0.3`
- Index and slice strings, and use common string methods
- Build readable output with f-strings

Every value has a type

A **data type** tells Python what kind of value something is and what operations make sense on it. Python checks types as the program runs.

Type	Name in code	Examples	What it holds
Integer	`int`	`42`, `-7`, `0`	Whole numbers
Float	`float`	`3.14`, `-0.5`, `2.0`	Numbers with a decimal point
String	`str`	`"Hello"`, `'a'`	Text, in quotes
Boolean	`bool`	`True`, `False`	Yes or no. Capitalized.

types.py

```
age = 15          # int
height = 5.8      # float
name = "Jordan"   # str
is_student = True # bool

print(type(age))
print(type(height))
print(type(name))
print(type(is_student))
```

Arithmetic and division

Python has two division operators. `/` always returns a float. `//` is floor division and returns a whole number. `%` is the remainder, which is useful for checking even and odd.

division.py

```
print(7 / 2)    # 3.5
print(7 // 2)   # 3
print(7 % 2)    # 1
print(2 ** 10)  # 1024
```

Converting between types

Input and file data often arrive as text. Convert with `int()`, `float()`, or `str()`. This is called **casting**.

casting.py

```
text = "42"
number = int(text)
print(number + 8)     # 50 (math)
print(text + "8")     # 428 (joins two strings)
print(float("3.5") * 2)
print(str(99) + " bottles")
```

Watch out

`int("3.7")` raises a `ValueError`. Convert to `float` first, then to `int` if you need to drop the decimal part: `int(float("3.7"))` gives `3`.

Floats are approximate

Computers store most decimal numbers in binary, which cannot represent 0.1 exactly. Small errors show up. Compare floats with a tolerance, not with `==`. Later you will use `math.isclose()`.

float_surprise.py

```
print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)
print(round(0.1 + 0.2, 2))
```

Strings

A string is an ordered sequence of characters. Each character has an **index**, starting at 0. Negative indexes count from the end. A **slice** `text[start:stop]` returns a part of the string; `stop` is not included.

strings.py

```
word = "Python"
print(word[0])      # P
print(word[-1])     # n
print(word[0:3])    # Pyt
print(word[2:])     # thon
print(len(word))    # 6
print(word.upper()) # PYTHON
print("py" in word.lower())  # True
```

Strings are **immutable**. You cannot change one character in place. You build a new string instead.

> **Watch out**
> `word[0] = "J"` raises `TypeError: 'str' object does not support item assignment`. Use `"J" + word[1:]` to build `"Jython"`.

Method	What it does	Example result
`.upper()` / `.lower()`	Changes letter case	`"hi".upper()` → `"HI"`
`.strip()`	Removes spaces at both ends	`"  hi  ".strip()` → `"hi"`
`.replace(old, new)`	Replaces every match	`"a-b".replace("-", "+")` → `"a+b"`
`.split(sep)`	Splits into a list	`"a,b".split(",")` → `["a", "b"]`
`.startswith(text)`	Checks the beginning	`"Python".startswith("Py")` → `True`

f-strings

Put an `f` before the opening quote and wrap any expression in braces. This is the cleanest way to mix text and values.

fstrings.py

```
score = 92
student = "Maya"
print(f"{student} scored {score}% on the test.")
print(f"Next goal: {score + 3}%")
```

Predict the output THINK FIRST

Work out each result by hand first.

```
print(7 / 2)
```

Answer: `3.5`

```
print(7 // 2)
```

Answer: `3`

```
print("5" * 3)
```

Answer: `555`
Multiplying a string repeats it.

```
print(bool(""))
```

Answer: `False`
An empty string is "falsy".

```
print(int("3.7"))
```

Answer: `ValueError`
A decimal string cannot be parsed as an int.

Practice

▼ **Exercise 1 — Full name**

Store a first name and a last name in two variables. Print the full name using an f-string, for example `Ada Lovelace`.

▼ **Exercise 2 — Clean and count**

Start with `raw = "  Python Is Fun  "`. Remove the extra spaces, make it lowercase, and print how many characters it has.

▼ **Hint**

Methods can be chained: `raw.strip().lower()`. Then call `len()` on the result.

Discussion

1. Why do you think `"5" + "5"` gives `"55"` while `5 + 5` gives `10`? What does that tell you about types?
2. Where in your daily life do you see numbers stored as text, and what would break if a program treated them as numbers?
3. If floats cannot represent 0.1 exactly, when should a bank program use integers (cents) instead?

Key terms

Data type

A category of value (int, float, str, bool) that determines what operations are valid on it.

int

Integer: a whole number with no decimal point.

float

Floating-point number: a number with a decimal point. Stored approximately.

str

String: an immutable sequence of characters written in quotes.

bool

Boolean: the value `True` or `False`. Named for George Boole.

type()

A built-in function that returns the type of a value, such as `<class 'int'>`.

Casting

Converting a value from one type to another, for example `int("42")`.

Index

The position of an item in a sequence. Counting starts at 0.

Slice

A part of a sequence written `seq[start:stop]`. The stop position is excluded.

Immutable

A value that cannot be changed after it is created. Strings, integers, and tuples are immutable.

f-string

A string literal beginning with `f` that embeds expressions in braces, such as `f"{x}"`.

Method

A function that belongs to an object and is called with a dot, such as `name.upper()`.

See all terms in the glossary →

UNIT 3 60 min

Boolean Logic and Truth Tables

Comparison operators, the logical operators and, or, not, precedence, and reading truth tables.

By the end of this unit you will be able to

- Write comparisons with `==`, `!=`, `<`, `>=` and explain the result
- Combine conditions with `and`, `or`, and `not`
- Build and read truth tables for a logic expression
- Explain short-circuit evaluation and why order matters
- Predict the result of an expression that mixes several operators

Comparisons produce booleans

A comparison asks a yes-or-no question and produces a `bool`. Every condition you will write in the next units is a comparison or a combination of them.

Operator	Meaning	Example	Result
`==`	equal to	`3 == 3`	`True`
`!=`	not equal to	`3 != 3`	`False`
`<` / `>`	less / greater than	`2 < 5`	`True`
`<=` / `>=`	less or equal / greater or equal	`5 >= 6`	`False`

Watch out

`=` assigns. `==` compares. Writing `if x = 5:` is a `SyntaxError`.

and, or, not

`and` is True only when **both** sides are True. `or` is True when **at least one** side is True. `not` flips the value.

logic.py

```
a = True
b = False
print(a and b)    # False
print(a or b)     # True
print(not a)      # False
print(5 > 3 and 2 == 2)   # True
```

Truth tables

A **truth table** lists every combination of inputs and the output for each. With two variables there are 2 × 2 = 4 rows. With three there are 8. Each extra variable doubles the rows. Use the table below to predict each column, then reveal the answers.

Truth tables for A and B INTERACTIVE

Predict each column first, then press **Reveal results**.

A	B	A and B	A or B	A != B (exclusive or)	not A
True	True	True	True	False	False
True	False	False	True	True	False
False	True	False	True	True	True
False	False	False	False	False	True

Note

The "exclusive or" column (called XOR) is True when exactly one input is True. It is the same as `!=` on two booleans.

Precedence

Operators run in a fixed order. From highest to lowest: comparisons, then `not`, then `and`, then `or`. Use parentheses whenever you are not sure. They make the code easier to read as well.

precedence.py

```
print(True or False and False)     # True: "and" runs first
print((True or False) and False)   # False
print(not True or True)            # True: "not" runs first
```

Short-circuit evaluation

Python stops evaluating as soon as the answer is known. In `x and y`, if `x` is False the whole expression is False and `y` is never run. In `x or y`, if `x` is True, `y` is skipped. This lets you write safe checks like `name is not None and name.strip() != ""`.

short_circuit.py

```
def check():
    print("check() was called")
    return True

print(False and check())   # check() never runs
print(True or check())    # check() never runs
```

Truthy and falsy values

Every value can be used as a condition. `0`, `0.0`, `""`, `[]`, and `None` are **falsy**. Everything else is **truthy**. Use this to write short checks like `if items:` to test for a non-empty list.

Predict the output THINK FIRST

Be careful with precedence and short-circuiting.

```
print(not 0)
```

Answer: `True`
0 is falsy, and `not` flips it.

```
print(3 == 3.0)
```

Answer: `True`
Int and float compare by value.

```
print("apple" < "banana")
```

Answer: `True`
Strings compare alphabetically, letter by letter.

```
print(1 < 2 < 3)
```

Answer: `True`
Python allows chained comparisons. It means `1 < 2 and 2 < 3`.

```
print(False and True or True)
```

Answer: `True`
`and` binds tighter, so it is `(False and True) or True`.

```
print(True or print("hi"))
```

Answer: `True`
The `or` already knows the answer, so `print("hi")` never runs.

Practice

▼ **Exercise 1 — Between 1 and 10**

Given `x = 7`, write one expression that is True only when `x` is from 1 to 10 inclusive. Print it.

▼ **Hint**

Python lets you chain comparisons. You can also write it with `and`.

▼ **Exercise 2 — NAND table**

Write a truth table for `not (A and B)`. Predict every row before you run the code, then check.

Discussion

1. Restaurants say "you may have soup or salad." Is that an inclusive or an exclusive or? How would you write each in Python?
2. Why is `a or b` safe to write when `a` is a check like `user is None`?
3. Can you write an expression equivalent to `A or B` using only `and` and `not`? Try it with a truth table.

Key terms

Boolean

A value that is either `True` or `False`.

Comparison operator

An operator that compares two values and returns a boolean: `==`, `!=`, `<`, `>`, `<=`, `>=`.

Logical operator

`and`, `or`, and `not`, which combine or invert boolean values.

Truth table

A table listing every combination of inputs and the resulting output of a logic expression.

XOR (exclusive or)

True when exactly one of two inputs is True. In Python, `a != b` on booleans.

Operator precedence

The fixed order in which operators are evaluated. Parentheses override it.

Short-circuit evaluation

Stopping a boolean expression as soon as its result is known, so later parts are never evaluated.

Truthy / falsy

Values that count as True or False in a condition. Falsy values include `0`, `""`, `[]`, and `None`.

Chained comparison

A comparison such as `1 < x < 10`, which Python evaluates as two comparisons joined by `and`.

See all terms in the glossary →

UNIT 4 60 min

Conditionals: Making Decisions

if, elif, and else; indentation as structure; nested decisions; the order of branches.

By the end of this unit you will be able to

- Write an `if` statement and explain why indentation matters
- Choose between a chain of `elif` branches and separate `if` statements
- Trace which branch runs for a given input
- Nest conditionals and keep them readable
- Avoid the common mistakes: `=` for `==`, missing colons, and wrong branch order

The if statement

An `if` statement runs a block of code only when its condition is True. The condition ends with a colon. The block under it is indented, usually by four spaces. Python uses indentation to know where the block ends.

if_basic.py

```
temperature = 92

if temperature > 85:
    print("Turn on the fan.")
    print("Drink some water.")

print("Done checking.")
```

The last line is not indented, so it runs no matter what the temperature is.

if, elif, else

Use `elif` ("else if") to test a list of alternatives. Python checks them in order and runs **only the first** branch that is True. Then it skips the rest. The `else` branch catches everything left over.

grades.py

```
score = 84

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"

print("Grade:", grade)
```

Trace: which branch runs? TRACE

```
score = 84

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"

print("Grade:", grade)
```

Step	Line run	Variables after the line	Output so far
1	1	`score = 84`	—
2	3	`score = 84`	—
3	5	`score = 84`	—
4	6	`score = 84, grade = 'B'`	—
5	12	`score = 84, grade = 'B'`	`Grade: B`

Watch out

Order matters. If you test `score >= 70` first, a score of 95 lands in the "C" branch. Put the strictest condition first, or use separate `if` statements.

Nested decisions

You can put a conditional inside another one. Each extra level adds four spaces. If you find yourself nesting three or more levels deep, consider a different structure, such as checking for the failing case first and returning or continuing early.

nested.py

```
age = 17
has_ticket = True

if has_ticket:
    if age >= 18:
        print("Enter, adult.")
    else:
        print("Enter, minor. Guardian required.")
else:
    print("Buy a ticket first.")
```

Combining conditions

Use `and`, `or`, and `not` from Unit 3 to combine tests into one condition. This is often cleaner than nesting.

combined.py

```
age = 17
has_ticket = True

if has_ticket and age >= 18:
    print("Enter, adult.")
elif has_ticket:
    print("Enter, minor.")
else:
    print("Buy a ticket first.")
```

Conditional expressions

For a simple choice between two values, Python has a one-line form: `value_if_true if condition else value_if_false`. Use it for short values only.

ternary.py

```
n = 7
print("even" if n % 2 == 0 else "odd")
```

Predict the output THINK FIRST

Predict which lines print. Pay attention to indentation and branch order.

```
x = 5
if x > 3:
    print("big")
```

Answer: `big`

```
x = 5
if x > 3:
    print("a")
if x > 1:
    print("b")
```

Answer: `a b`
Two separate `if` statements are both checked.

```
x = 5
if x > 3:
    print("a")
elif x > 1:
    print("b")
```

Answer: `a`
Only the first true branch runs.

```
x = 0
if x:
    print("yes")
else:
    print("no")
```

Answer: `no`
0 is falsy.

```
print("even" if 4 % 2 == 0 else "odd")
```

Answer: `even`

Practice

▼ **Exercise 1 — Adult or minor**

Given `age = 16`, print `Adult` if the age is 18 or more, otherwise print `Minor`.

▼ **Exercise 2 — Ticket price**

A movie ticket costs $12. Students (age 17 and under) pay $8. Seniors (age 65 and over) pay $9. Given `age = 70`, print the price.

▼ **Hint**

Check the age groups in an order that makes each branch correct.

Discussion

1. Rewrite the grades example with separate `if` statements instead of `elif`. What breaks, and why?
2. When is nesting clearer than combining with `and`? When is it worse?
3. Give an example of a real-world rule that has an `else` branch you would not want to forget.

Key terms

Condition
An expression that evaluates to True or False and decides whether a block runs.

Branch
One path through a conditional. Each `if`, `elif`, and `else` defines a branch.

Indentation
Leading spaces that show which lines belong to a block. Python requires consistent indentation.

Block
A group of statements that run together, introduced by a colon and indented beneath a statement.

elif
Short for "else if". Tests another condition only if the previous ones were False.

Nested conditional
A conditional placed inside another conditional's block.

Conditional expression
A one-line choice written `a if condition else b`.

See all terms in the glossary →

UNIT 5 70 min

Loops: Repeating Work

for loops over ranges, while loops with conditions, accumulators, break, and continue.

By the end of this unit you will be able to

- Write a `for` loop over a `range()` and explain what the loop variable holds
- Write a `while` loop and identify the update that makes it end
- Use an accumulator to build up a total
- Use `break` and `continue` to change the flow of a loop
- Recognize and avoid an infinite loop

Why loops

A loop repeats a block of code. Without loops you would copy a line 1,000 times. With them, the program repeats the same logic for each item, each number, or until a condition changes.

for loops and range()

A `for` loop visits each item in a sequence, one at a time. `range(stop)` produces the numbers from 0 up to but **not including** `stop`. `range(start, stop)` starts at `start`. `range(start, stop, step)` counts by `step`.

for_range.py

```
for i in range(3):
    print("Count:", i)

for n in range(2, 10, 3):
    print("Stepped:", n)
```

Trace: summing 1 to 3 TRACE

```
total = 0
for i in range(1, 4):
    total = total + i
print(total)
```

Step	Line run	Variables after the line	Output so far
1	1	`total = 0`	`—`
2	2	`total = 0, i = 1`	`—`
3	3	`total = 1, i = 1`	`—`
4	2	`total = 1, i = 2`	`—`
5	3	`total = 3, i = 2`	`—`
6	2	`total = 3, i = 3`	`—`
7	3	`total = 6, i = 3`	`—`
8	2	`total = 6, i = 3`	`—`
9	4	`total = 6, i = 3`	`6`

Note

`range(1, 4)` gives 1, 2, 3. It does not include 4. That is the most common off-by-one mistake in this unit.

while loops

A `while` loop keeps running as long as its condition is True. It is checked before each pass. You must change something inside the loop, or it never stops.

countdown.py

```
count = 3
while count > 0:
    print(count)
    count = count - 1
print("Liftoff!")
```

Trace: the countdown TRACE

```
1  count = 3
2  while count > 0:
3      print(count)
4      count = count - 1
5  print("Liftoff!")
```

Step	Line run	Variables after the line	Output so far
1	1	`count = 3`	—
2	2	`count = 3`	—
3	3	`count = 3`	`3`
4	4	`count = 2`	`3`
5	2	`count = 2`	`3`
6	3	`count = 2`	`3 | 2`
7	4	`count = 1`	`3 | 2`
8	2	`count = 1`	`3 | 2`
9	3	`count = 1`	`3 | 2 | 1`
10	4	`count = 0`	`3 | 2 | 1`
11	2	`count = 0`	`3 | 2 | 1`
12	5	`count = 0`	`3 | 2 | 1 | Liftoff!`

Watch out

Infinite loops. If the condition never becomes False, the program runs forever. Common causes: forgetting to update the variable, or using `while True:` with no `break`. If you run one in this site, press the browser's reload button.

break and continue

- `break` leaves the loop immediately.
- `continue` skips the rest of this pass and moves to the next item.

break_continue.py

```
for n in range(1, 8):
    if n == 3:
        continue       # skip 3
    if n == 6:
        break          # stop before 6
    print(n)
```

Loops and conditions together

Loops and `if` statements combine freely. A common pattern is a loop that searches for something and stops when it finds it.

search.py

```
scores = [72, 95, 88, 61]
for s in scores:
    if s >= 90:
        print("First score at or above 90:", s)
        break
else:
    print("No score reached 90.")
```

Tip

The `else` on a loop runs only when the loop finished *without* hitting `break`. It is unusual, so label it clearly in your code.

Predict the output THINK FIRST

Trace each loop in your head before you run it.

```
for i in range(3):
    print(i)
```

Answer: `0 1 2`

`range(3)` stops before 3.

```
print(list(range(2, 10, 3)))
```

Answer: `[2, 5, 8]`

```
for i in range(5):
    if i == 2:
        break
    print(i)
```

Answer: `0 1`

```
for i in range(4):
    if i % 2 == 0:
        continue
    print(i)
```

Answer: `1 3`

```
n = 0
while n < 3:
    n = n + 1
    print(n)
```

Answer: `1 2 3`

```
print(sum(range(1, 5)))
```

Answer: `10`
1 + 2 + 3 + 4.

Practice

▼ **Exercise 1 — Divisible by three**

Print every number from 1 to 20 that is divisible by 3.

▼ **Hint**

Use `%`. A number is divisible by 3 when `n % 3 == 0`.

▼ **Exercise 2 — Running total**

Starting at `total = 0`, add 7 to `total` each time, using a `while` loop. Stop as soon as `total` is 50 or more. Print the final total.

Discussion

1. When would you choose `for` and when `while`? Give one example of each from everyday life.
2. The trace shows line 2 running one more time than the body. Why does the check need to run one extra time?
3. What is the risk of using `while True:`, and how does a `break` reduce it?

Key terms

Loop

A statement that repeats a block of code.

Iteration

One pass through the body of a loop.

for loop

A loop that assigns each item of a sequence to a variable in turn.

while loop

A loop that repeats while its condition is True. The condition is checked before each pass.

range()

A built-in that produces a sequence of integers from `start` up to but not including `stop`.

Accumulator

A variable that collects a running result, such as a total, across loop iterations.

break

A statement that exits the nearest loop immediately.

continue

A statement that skips the rest of the current iteration and moves to the next.

Infinite loop

A loop whose condition never becomes False, so it never ends unless interrupted.

Off-by-one error

A bug caused by a range or index that is one too high or too low.

See all terms in the glossary →

UNIT 6 70 min

Functions

Define reusable code with def, pass inputs, return results, and understand local versus global scope.

By the end of this unit you will be able to

- Define a function with `def` and call it
- Distinguish parameters from arguments
- Return a value with `return` and explain what happens when there is no `return`
- Use default and keyword arguments
- Explain why variables inside a function are local

Why functions

A **function** is a named block of code you can run as many times as you need. Functions make programs shorter, easier to test, and easier to change, because the logic lives in one place. You have been calling functions since Unit 1: `print()`, `range()`, and `int()` are all functions.

Defining and calling

greet.py

```
def greet(name):
    print(f"Hello, {name}!")

greet("Ada")
greet("Ben")
```

- `def` starts the definition. The name follows, then parentheses, then a colon.
- **Parameters** are the names listed in the definition (`name`).
- **Arguments** are the values you pass when calling (`"Ada"`).

Returning values

`return` sends a value back to the caller and ends the function. A function without a `return` gives back `None`, Python's "no value" marker. Use `return` when the caller needs the result; use `print()` when the goal is only to show something.

Trace: a function with a return value TRACE

```
1  def add_tax(price, rate=0.07):
2      tax = price * rate
3      return price + tax
4
5  total = add_tax(100)
6  print(total)
```

Step	Line run	Variables after the line	Output so far
1	1	`add_tax = function`	—
2	2	`add_tax = function, price = 100, rate = 0.07, tax = 7.0`	—
3	3	`add_tax = function, price = 100, rate = 0.07, tax = 7.0`	—
4	5	`add_tax = function, total = 107.0`	—
5	6	`add_tax = function, total = 107.0`	`107.0`

Default and keyword arguments

A parameter can have a default value. Callers may leave it out. You can also pass arguments by name, which makes calls easier to read.

defaults.py

```
def greet(name="friend", punctuation="!"):
    print(f"Hi {name}{punctuation}")

greet()
greet("Maya")
greet(punctuation="?", name="Sam")
```

Scope: where variables live

A variable created inside a function is **local**. It exists only during that call. A variable created at the top level of a file is **global**. A function can *read* a global, but assigning to a name inside a function creates a new local variable instead of changing the global one.

scope.py

```
total = 10

def try_to_change():
    total = 99          # creates a new local variable
    print("inside:", total)

try_to_change()
print("outside:", total)
```

> **Watch out**
> Using a local variable outside its function raises `NameError`. Return the value instead of relying on a shared variable.

Good function habits

- Give each function one clear job and a verb name: `calculate_tax`, not `stuff`.
- Add a short docstring, a string as the first line of the body, that says what the function does.
- Prefer passing values in and returning results over changing globals.

docstring.py

```
def is_even(n):
    """Return True if n is divisible by 2."""
    return n % 2 == 0

print(is_even(4))
print(is_even(7))
```

Predict the output THINK FIRST

Remember: a missing `return` gives `None`.

```
def double(x):
    return x * 2

print(double(4))
```

Answer: `8`

```
def hi(name="friend"):
    print("Hi", name)

hi()
```

Answer: `Hi friend`

```
n = 5
def bump(n):
    n = n + 1
    return n

bump(n)
print(n)
```

Answer: `5`
The function returned 6 but nothing kept it, and the global `n` was never changed.

```
def check(x):
    if x > 0:
        return "positive"

print(check(-1))
```

Answer: `None`
No `return` ran, so the function gives back `None`.

Practice

▼ **Exercise 1 — Celsius to Fahrenheit**

Write a function `c_to_f(celsius)` that returns the Fahrenheit value (`celsius * 9 / 5 + 32`). Print the result for 100 and for 0.

▼ **Exercise 2 — Largest of three**

Write `largest(a, b, c)` that returns the biggest of the three numbers **without** using the built-in `max()`. Test it with 4, 9, and 2.

▼ **Hint**

Start by assuming `a` is the biggest, then compare it to `b` and `c` with `if` statements.

Discussion

1. Why does `scope.py` print 99 inside and 10 outside? What would make a beginner expect the outside value to change?
2. When would you want a default argument, and when would you rather require the caller to supply the value?
3. If you wrote one function that does three jobs, how would you split it?

Key terms

Function

A named, reusable block of code, defined with `def` and run by calling it.

Parameter

A variable name in a function definition that receives a value when the function is called.

Argument

The actual value passed to a function when it is called.

return

A statement that ends a function and sends a value back to the caller.

None

Python's special value meaning "no value". Functions without a `return` produce it.

Default argument

A parameter with a preset value, such as `rate=0.07`, used when the caller does not supply one.

Keyword argument

An argument passed by name, such as `greet(name="Sam")`.

Local variable

A variable that exists only inside the function where it was created.

Global variable

A variable defined at the top level of a program, visible to the code that follows it.

Docstring

A string literal placed as the first line of a function body to document what it does.

See all terms in the glossary →

UNIT 7 70 min

Lists

Ordered collections: indexing, slicing, changing and growing a list, sorting, and the aliasing trap.

By the end of this unit you will be able to

- Create lists and access items by index, including negative indexes
- Slice a list and explain what the slice includes
- Add, remove, and change items with `append`, `insert`, `remove`, `pop`, and assignment
- Explain why a list is mutable and what aliasing means
- Loop over a list and build a new list from it

What a list is

A **list** is an ordered collection of values, written in square brackets and separated by commas. The items can be any type, and they can be mixed. Lists are **mutable**: you can change them after they are created.

lists_basics.py

```
scores = [72, 95, 88, 61]
names = ["Ana", "Ben", "Cy"]
mixed = [1, "two", 3.0, True]

print(scores)
print(len(names))
print(type(mixed))
```

Indexing and slicing

Indexes start at 0. `list[-1]` is the last item. A slice `list[start:stop]` returns a new list that includes `start` and excludes `stop`.

indexing.py

```
letters = ["a", "b", "c", "d", "e"]
print(letters[0])     # a
print(letters[-1])    # e
print(letters[1:4])   # ['b', 'c', 'd']
print(letters[:2])    # first two
print(letters[3:])    # from index 3 to the end
```

Watch out

Asking for an index that does not exist raises `IndexError: list index out of range`. For a list of 5 items, the valid indexes are 0 to 4 (or -5 to -1).

Changing a list

Action	Code	Effect
Add to the end	`items.append(x)`	Adds one item at the end
Insert at a position	`items.insert(0, x)`	Puts x at index 0; everything shifts right
Remove by value	`items.remove(x)`	Removes the first match; error if absent
Remove by position	`items.pop(i)`	Removes and returns the item at i (default: last)
Replace	`items[2] = x`	Overwrites the item at index 2

changing.py

```
fruit = ["apple", "banana"]
fruit.append("cherry")
fruit.insert(0, "avocado")
fruit[1] = "blueberry"
removed = fruit.pop()
print(fruit)
print("removed:", removed)
```

Methods like `append` change the list in place and return `None`. Do not write `x = x.append(1)`. You would overwrite the list with `None`.

Sorting

`sorted(items)` returns a new sorted list and leaves the original alone. `items.sort()` sorts in place and returns `None`. Add `reverse=True` for descending order.

Aliasing: two names, one list

Assigning a list to a new name does **not** copy it. Both names point to the same list. Changing one changes the other. Use `.copy()` or `list(original)` when you need an independent copy.

Trace: aliasing TRACE

```
1  a = [1, 2]
2  b = a
3  b.append(3)
4  print(a)
```

Step	Line run	Variables after the line	Output so far
1	1	`a = [1, 2]`	—
2	2	`a = [1, 2], b = [1, 2]`	—
3	3	`a = [1, 2, 3], b = [1, 2, 3]`	—
4	4	`a = [1, 2, 3], b = [1, 2, 3]`	`[1, 2, 3]`

Looping over a list

A `for` loop visits each item in order. Use `enumerate()` when you also need the index.

loop_lists.py

```
names = ["Ana", "Ben", "Cy"]
for name in names:
    print("Hi", name)

for i, name in enumerate(names):
    print(i, name)
```

Tip

Do not add or remove items from a list while looping over it. Build a new list instead: `evens = [n for n in nums if n % 2 == 0]` is the compact form of a loop with `append`.

Predict the output THINK FIRST

Watch for in-place methods and aliasing.

```
nums = [10, 20, 30]
print(nums[1])
```

Answer: `20`

```
nums = [10, 20, 30]
print(nums[-1])
```

Answer: `30`

```
nums = [10, 20, 30]
nums.append(40)
print(len(nums))
```

Answer: `4`

```
print([1, 2, 3, 4, 5][1:4])
```

Answer: `[2, 3, 4]`

```
letters = ["a", "b"]
letters.insert(0, "z")
print(letters)
```

Answer: `['z', 'a', 'b']`

```
x = [3, 1, 2]
print(x.sort())
```

Answer: `None`
`sort()` changes the list and returns nothing.

```
print(5 in [1, 2, 3])
```

Answer: `False`

```
nums = [1, 2, 3]
nums[5] = 9
```

Answer: `IndexError`

Practice

▼ **Exercise 1 — Average score**

Given `scores = [72, 95, 88, 61]`, print the average. Use `sum()` and `len()`.

▼ **Exercise 2 — Squares**

Build a list called `squares` containing the squares of 1 through 5 (1, 4, 9, 16, 25). Use a loop and `append`. Print the list.

Discussion

1. Why do you think Python makes lists mutable instead of returning a new list for every change? What does that cost in clarity?
2. Predict: if two friends share one shopping list in a file and one of them deletes an item, what happens to the other? How is that like aliasing?
3. When is `sorted()` the better choice than `sort()`?

Key terms

List

An ordered, mutable collection of values written in square brackets.

Index

The position of an item in a list, starting at 0. Negative indexes count from the end.

Slice

A new list made from a range of positions, written `items[start:stop]`.

Mutable

A value that can be changed after it is created. Lists and dictionaries are mutable.

Method

A function attached to an object. `append`, `insert`, `pop`, and `sort` are list methods.

Aliasing

When two names refer to the same object, so a change through one name is visible through the other.

enumerate()

A built-in that gives each item of a sequence together with its index, for use in a loop.

IndexError

The error raised when an index is outside the range of a list.

List comprehension

A compact expression that builds a list from a loop, such as `[n * n for n in range(5)]`.

See all terms in the glossary →

UNIT 8 60 min

Tuples and Sets

Fixed-size tuples for grouped values, and sets for unique items with fast membership tests.

By the end of this unit you will be able to

- Create tuples and explain why they are immutable
- Unpack a tuple into separate variables and return several values from a function
- Create sets, explain why they hold no duplicates, and use union, intersection, and difference
- Choose between a list, a tuple, and a set for a task

Tuples: fixed groups

A **tuple** is an ordered collection written with parentheses. Like a list, it is indexed and sliced. Unlike a list, it is **immutable**: once created, you cannot add, remove, or replace items. Use a tuple when the group should not change, such as an (x, y) coordinate or an RGB color.

tuples.py

```
point = (4, 7)
color = (255, 128, 0)
print(point[0])
print(len(color))
```

Watch out

A single-item tuple needs a trailing comma: `(5,)` is a tuple, but `(5)` is just the number 5.

Unpacking

You can assign each item of a tuple to its own name in one line. This is called **unpacking**. The number of names must match the number of items.

unpacking.py

```
x, y = (3, 4)
print(x + y)

def min_max(nums):
    return min(nums), max(nums)

low, high = min_max([7, 2, 9, 4])
print(low, high)
```

Python functions return multiple values by returning a tuple. The caller unpacks it. This is the most common reason you will see tuples in real code.

Sets: unique items

A **set** is an unordered collection of unique values, written with curly braces. Adding a duplicate does nothing. Checking whether an item is in a set is very fast, even for large sets.

sets.py

```
visited = {1, 2, 2, 3, 3, 3}
print(visited)
print(len(visited))
visited.add(4)
print(4 in visited)
```

> **Watch out**
> `{}` creates an empty **dictionary**, not a set. Use `set()` for an empty set. Also, the order of items in a set is not guaranteed, so do not rely on it.

Set operations

Operation	Symbol	Method	Meaning
Union	`a | b`	`a.union(b)`	Items in either set

Operation	Symbol	Method	Meaning
Intersection	`a & b`	`a.intersection(b)`	Items in both sets
Difference	`a - b`	`a.difference(b)`	Items in a but not in b

set_ops.py

```
math_class = {"Ana", "Ben", "Cy"}
art_class = {"Ben", "Dee"}

print(math_class | art_class)
print(math_class & art_class)
print(math_class - art_class)
```

Choosing a collection

Need	Use	Why
An ordered sequence that changes	list	Indexable, mutable, allows duplicates
A fixed group, such as coordinates	tuple	Immutable; safe to pass around
Unique items, fast membership tests	set	Duplicates removed automatically
Labels that map to values	dict (Unit 9)	Look up by key instead of position

Tip

To remove duplicates from a list while keeping it a list: `unique = list(set(items))`. The order may change, so use this only when order does not matter.

Predict the output THINK FIRST

Watch for the single-item tuple and for the empty set.

```
print((5))
```

Answer: `5`
Parentheses alone do not make a tuple.

```
print((5,))
```

Answer: `(5,)`

```
t = (1, 2)
t[0] = 9
```

Answer: `TypeError`
Tuples cannot be changed.

```
x, y = (3, 4)
print(x + y)
```

Answer: `7`

```
print({1, 2, 2, 3})
```

Answer: `{1, 2, 3}`

```
print({1, 2} | {2, 3})
```

Answer: `{1, 2, 3}`

```
print({1, 2, 3} & {2, 3, 4})
```

Answer: `{2, 3}`

```
print(len(set("banana")))
```

Answer: `3`
The unique letters are b, a, n.

```
print({} == set())
```

Answer: `False`
`{}` is an empty dictionary.

Practice

▼ **Exercise 1 — Shared clubs**

Two clubs have these members: `chess = ["Ana", "Ben", "Cy", "Ben"]` and `robotics = ["Ben", "Dee", "Ana"]`. Print the names that appear in **both** clubs, without duplicates.

▼ **Hint**

Convert each list to a set, then use `&`.

▼ **Exercise 2 — Unpack a return**

Write a function `stats(nums)` that returns a tuple of `(smallest, largest, count)`. Call it with `[4, 8, 1, 6]` and unpack the result into three variables. Print each.

Discussion

1. Why might a programmer choose a tuple over a list even when both would work? What is the benefit of "can't change it"?
2. A set throws away duplicates and order. When is that a feature, and when is it a bug?
3. Pick one real data source (a roster, a list of coordinates, a log of website visits) and say which collection type you would use and why.

Key terms

Tuple
An immutable, ordered collection written in parentheses, such as `(4, 7)`.

Immutable
Cannot be changed after creation. Tuples, strings, and numbers are immutable.

Unpacking
Assigning each item of a sequence to a separate name in one statement, such as `x, y = point`.

Set
An unordered collection of unique values, written in curly braces or created with `set()`.

Union
The set of items in either of two sets. Written `a | b`.

Intersection
The set of items in both sets. Written `a & b`.

Difference
The items in the first set that are not in the second. Written `a - b`.

Membership test
Checking whether a value is in a collection with the `in` operator.

See all terms in the glossary →

UNIT 9 75 min

Dictionaries

Store values under named keys: create, read, update, safely look up, loop over, and nest dictionaries.

By the end of this unit you will be able to

- Create a dictionary and read, add, and update values by key
- Use `.get()` to look up a key that might be missing
- Loop over keys, values, and key-value pairs with `.items()`
- Build a dictionary by counting items in a loop
- Work with a dictionary nested inside another dictionary

Key-value pairs

A **dictionary** maps **keys** to **values**. Instead of remembering that item 2 is Ben's score, you look up `scores["Ben"]`. Keys must be immutable (strings and numbers are the usual choices). Values can be anything.

dict_basics.py

```
student = {
    "name": "Maya",
    "age": 16,
    "grades": [91, 88, 95]
}

print(student["name"])
print(student["grades"][0])
```

Adding and changing values

Assigning to a key that exists updates it. Assigning to a new key adds it.

dict_update.py

```
scores = {"Ana": 90}
scores["Ben"] = 85      # new key: added
scores["Ana"] = 95      # existing key: updated
print(scores)
```

Safe lookups with get()

Reading a missing key raises `KeyError`. `.get(key, default)` returns the default instead, which is the safe choice when the key might not exist.

get_safe.py

```
prices = {"apple": 0.50, "pear": 0.75}
print(prices.get("apple", 0))
print(prices.get("kiwi", 0))
```

> **Watch out**
> A missing key with `prices["kiwi"]` raises `KeyError: 'kiwi'`. Use `.get()` when the key is not guaranteed to exist.

Looping over a dictionary

`.keys()` gives the keys, `.values()` gives the values, and `.items()` gives both as pairs. Looping over the dictionary directly visits the keys.

Trace: loop over items TRACE

```
grades = {"Ana": 90}
grades["Ben"] = 85
grades["Ana"] = 95
for name, score in grades.items():
    print(name, score)
```

Step	Line run	Variables after the line	Output so far
1	1	`grades = {'Ana': 90}`	—
2	2	`grades = {'Ana': 90, 'Ben': 85}`	—
3	3	`grades = {'Ana': 95, 'Ben': 85}`	—
4	4	`grades = {'Ana': 95, 'Ben': 85}, name = 'Ana', score = 95`	—
5	5	`grades = {'Ana': 95, 'Ben': 85}, name = 'Ana', score = 95`	`Ana 95`
6	4	`grades = {'Ana': 95, 'Ben': 85}, name = 'Ben', score = 85`	`Ana 95`
7	5	`grades = {'Ana': 95, 'Ben': 85}, name = 'Ben', score = 85`	`Ana 95 | Ben 85`

Counting with a dictionary

A very common pattern is counting how often each item appears. Start empty, then for each item, use `.get(item, 0) + 1` to add one to its count.

counter.py

```
words = "the cat saw the dog and the bird".split()
counts = {}
for w in words:
    counts[w] = counts.get(w, 0) + 1
print(counts)
print("'the' appears", counts["the"], "times")
```

Nested dictionaries

A dictionary can hold other dictionaries or lists. Chain the keys to reach deeper values. Real data (JSON from web APIs) looks exactly like this.

nested.py

```
class_roster = {
    "period_1": {"teacher": "Ms. Lee", "students": ["Ana", "Ben"]},
    "period_2": {"teacher": "Mr. Cruz", "students": ["Cy", "Dee", "Eli"]}
}

print(class_roster["period_2"]["teacher"])
print(len(class_roster["period_2"]["students"]))
```

Predict the output THINK FIRST

Keys decide what is found. Watch for duplicates and for missing keys.

```
d = {"a": 1}
print(d["a"])
```

Answer: `1`

```
d = {"a": 1}
print(d["b"])
```

Answer: `KeyError`

```
print({"x": 1}.get("y", 0))
```

Answer: `0`

```
d = {"k": 2}
d["k"] = 5
print(d)
```

Answer: `{'k': 5}`

```
print(len({"a": 1, "a": 2}))
```

Answer: `1`

The second "a" replaces the first. Keys are unique.

```
print(1 in {"a": 1})
```

Answer: `False`
`in` checks keys, not values.

```
d = {"p": {"age": 3}}
print(d["p"]["age"])
```

Answer: `3`

Practice

▼ **Exercise 1 — Inventory**

Start with `stock = {"pencil": 4, "pen": 2}`. Add 3 pens, add 5 erasers (a new key), then print the whole dictionary.

▼ **Exercise 2 — Most common**

Using the counts from the counter example, find the word with the highest count and print it. Test with the sentence `"a b a c a b"`.

▼ **Hint**

Loop over `counts.items()` and keep track of the best word and count seen so far.

Discussion

1. When is a list of pairs a better choice than a dictionary, and when is it worse?
2. Why do you think dictionary keys must be immutable? What would go wrong if a key could change after it was stored?
3. Think of a real system you use (a school portal, a game, a store app). What are its keys and values?

Key terms

Dictionary

A collection that maps unique keys to values, written in curly braces with `key: value` pairs.

Key

The label used to look up a value in a dictionary. Keys must be immutable and unique.

Value

The data stored under a key. Values can be any type.

.get()

A dictionary method that returns the value for a key, or a default if the key is missing.

.items()

A dictionary method that yields key-value pairs, usually unpacked in a loop as `for k, v in d.items()`.

KeyError

The error raised when reading a key that is not in the dictionary.

Nested dictionary

A dictionary that contains another dictionary or list as a value.

Counter pattern

Using a dictionary to count occurrences: `counts[item] = counts.get(item, 0) + 1`.

See all terms in the glossary →

UNIT 10 60 min

Libraries and Modules

Import the standard library, read its documentation, and split your own code into modules.

By the end of this unit you will be able to

- Import a module and call its functions with dot notation
- Use `from ... import` and `import ... as` and explain when each is appropriate
- Find a function's documentation with `help()` and `dir()`
- Explain what the standard library includes and what `pip` adds
- Create a module file and import it from another file

Modules and libraries

A **module** is a file of Python code that can be imported. A **library** is a collection of modules. Python ships with a large **standard library**, so many common tasks need no installation. Anything else you install with `pip`, which is covered in the NumPy and pandas units.

Importing

imports.py

```
import math
import statistics

print(math.sqrt(16))
print(math.pi)
print(math.floor(3.7))
print(statistics.mean([2, 4, 6]))
```

After `import math`, everything in the module is reached with a dot: `math.sqrt`. The name `math` comes first so you always know where a function lives.

import_styles.py

```
from math import ceil, factorial
print(ceil(2.1))
print(factorial(5))

import math as m
print(m.hypot(3, 4))
```

Tip

Use `import math` when many names come from one module. Use `from math import ceil` when you need only one or two names. An alias such as `as m` is common for long names; pandas and NumPy use `pd` and `np` by convention.

Finding out what a module offers

discover.py

```
import math

print(len(dir(math)))          # how many names the module has
print(math.sqrt.__doc__)       # the documentation string
```

In an interactive environment, `help(math.sqrt)` prints the same documentation. Reading the documentation is a skill you will use every day. Get in the habit of checking the docs before guessing.

Some useful standard-library tools

Module	What it gives you	Example
`math`	Square root, rounding, trig, constants	`math.sqrt(9)` → 3.0
`statistics`	Mean, median, standard deviation	`statistics.median([1, 3, 9])` → 3
`random`	Random choices and numbers	`random.choice(["a", "b"])`

Module	What it gives you	Example
`datetime`	Dates and times	`datetime.date.today()`
`csv` / `json`	Read and write common data files	`json.loads(text)`

> **Note**
> Random results differ on every run. To get the same "random" numbers each time while you test, call `random.seed(42)` first. Seeding is standard practice in data science and machine learning.

Writing your own module

Any `.py` file is a module. Put a helpful function in its own file and import it from your main program. Two files are needed, so this example is shown as code to save on your computer rather than in the browser sandbox.

greetings.py (module file)

```
def hello(name):
    """Return a friendly greeting."""
    return f"Hello, {name}!"

if __name__ == "__main__":
    # Runs only when you run this file directly, not when it is imported.
    print(hello("test"))
```

main.py (in the same folder)

```
from greetings import hello

print(hello("Ada"))
```

The `if __name__ == "__main__":` line lets a file work both as a script and as a module. Python sets `__name__` to `"__main__"` only when the file is run directly.

Watch out

Do not name your file the same as a standard-library module (for example `random.py` or `math.py`). Python will import your file instead of the library and the program will fail in confusing ways.

Predict the output THINK FIRST

Remember what `round()` does with exactly .5, and that `math` functions return floats.

```
import math
print(math.floor(-2.5))
```

Answer: `-3`

```
import math
print(math.ceil(2.1))
```

Answer: `3`

```
from math import pow
print(pow(2, 3))
```

Answer: `8.0`

```
import math
print(math.factorial(5))
```

Answer: `120`

```
print(round(2.5))
```

Answer: `2`
Python rounds .5 to the nearest even number ("banker's rounding").

```
print(round(3.5))
```

Answer: `4`

```
import math
print(math.hypot(3, 4))
```

Answer: `5.0`

```
print(abs(-7))
```

Answer: `7`

Practice

▼ **Exercise 1 — Hypotenuse two ways**

For a right triangle with legs 5 and 12, compute the hypotenuse once with `math.sqrt` and once with `math.hypot`. Print both results.

▼ **Exercise 2 — Build a module**

Create `shapes.py` with a function `circle_area(radius)` that returns `math.pi * radius ** 2`. In a second file, import it and print the area of a circle with radius 2. Write both files on your computer.

▼ **Hint**

The module file needs `import math` at the top.

Discussion

1. Why do you think Python includes a large standard library instead of making everyone install packages? What are the trade-offs?
2. What is the risk of a file named `random.py` in your project folder? How would you find the cause of the error?
3. When you see `import numpy as np` in someone else's code, what do you expect the rest of the code to look like?

Key terms

Module

A Python file (`.py`) whose functions and variables can be imported into another program.

Library

A collection of modules that provides a set of related tools, such as the standard library, NumPy, or pandas.

Standard library

The modules that come with every Python installation, such as `math`, `random`, and `json`.

import

A statement that loads a module so its names can be used.

Alias

A shorter name given to a module on import, such as `import numpy as np`.

pip

The package installer for Python. It downloads third-party libraries from the Python Package Index.

Seed

A starting value for a random number generator. The same seed produces the same sequence every run.

__name__

A built-in variable. It equals `"__main__"` when a file is run directly, and the module name when imported.

See all terms in the glossary →

UNIT 11 60 min

Errors and Exceptions

Read a traceback, catch expected failures with try/except, and raise errors when input is wrong.

By the end of this unit you will be able to

- Read a traceback and identify the exception type, message, and line
- Use `try` / `except` to handle a failure you expect
- Catch a specific exception rather than everything, and explain why
- Use `else` and `finally` with a try block
- Raise an exception from your own function when input is invalid

An exception stops the program

When Python hits a problem it cannot resolve, it raises an **exception**. If nothing catches it, the program stops and prints a **traceback**. Read a traceback from the bottom up: the last line gives the exception type and message, and the lines above show the path that led to it.

traceback.py

```
def average(values):
    return sum(values) / len(values)

print(average([]))
```

The bottom line reads `ZeroDivisionError: division by zero`. The line above shows that `average` failed on `sum(values) / len(values)`, because the list was empty.

try and except

Put code that might fail inside `try`. If it raises an exception, Python jumps to the matching `except` block instead of stopping. Use this for failures you expect and can recover from, such as a user typing text where a number belongs.

safe_int.py

```
def safe_int(text):
    try:
        return int(text)
    except ValueError:
        return None

print(safe_int("42"))
print(safe_int("forty"))
```

Catch specific exceptions

Name the exception you expect. A bare `except:` or `except Exception:` also catches bugs you did not mean to handle, such as a typo in a variable name, and hides them. Catch the smallest set that makes sense, and handle each one differently when needed.

specific.py

```
def read_score(items, index):
    try:
        value = int(items[index])
    except IndexError:
        return "no such position"
    except ValueError:
        return "not a number"
    return value

print(read_score(["90", "x"], 0))
print(read_score(["90", "x"], 1))
print(read_score(["90", "x"], 5))
```

Watch out

A bare `except:` catches even `KeyboardInterrupt`, which is how a user stops a program. Avoid it. Catch named exceptions.

else and finally

- `else` runs only if the `try` block finished without an exception. Keep it for the code that depends on success.
- `finally` runs every time, whether or not an exception happened. Use it to clean up, such as closing a file.

else_finally.py

```
try:
    number = int("15")
except ValueError:
    print("bad input")
else:
    print("doubled:", number * 2)
finally:
    print("check finished")
```

Raising your own exceptions

`raise` starts an exception on purpose. It is how a function reports that the input is wrong. The caller can then catch it, or let it stop the program. Choose a standard type that matches the problem: `ValueError` for a bad value, `TypeError` for a wrong type.

raise.py

```
def set_age(age):
    if not isinstance(age, int):
        raise TypeError("age must be a whole number")
    if age < 0:
        raise ValueError("age cannot be negative")
    return age

try:
    set_age(-4)
except ValueError as error:
    print("Rejected:", error)
```

Ask forgiveness or check first

Python programmers often try the operation and handle the failure ("easier to ask forgiveness than permission"). The other style checks first with an `if`. Both are valid. Use the check when it is simple and clear, such as `if divisor != 0`. Use `try` when the check is complicated or the failure can happen in many places.

Common exceptions

Exception	Usually means	Example that raises it
`NameError`	Name not assigned	`print(total)` before `total = 0`
`TypeError`	Operation on the wrong type	`"5" + 5`
`ValueError`	Right type, bad value	`int("x")`
`IndexError`	Position out of range	`[1, 2][5]`
`KeyError`	Key not in dictionary	`{"a": 1}["b"]`
`ZeroDivisionError`	Division by zero	`1 / 0`
`FileNotFoundError`	File does not exist	`open("missing.txt")`

Predict the output THINK FIRST

Trace the flow: which block runs, and does the program keep going?

```
print(int("12") + 1)
```

Answer: `13`

```
try:
    print(1 / 0)
except ZeroDivisionError:
    print("caught")
```

Answer: `caught`

```
try:
    x = int("a")
except ValueError:
    print("bad")
else:
    print("ok")
```

Answer: `bad`
The `else` block is skipped because an exception happened.

```
try:
    print("a")
finally:
    print("done")
```

Answer: `a done`

```
try:
    raise ValueError("no")
except Exception as e:
    print(type(e).__name__, e)
```

Answer: `ValueError no`

Practice

▼ **Exercise 1 — Safe divide**

Write `safe_divide(a, b)` that returns `a / b`, or `None` when `b` is zero. Use `try` / `except`. Test it with `(10, 2)` and `(10, 0)`.

▼ **Exercise 2 — Validate a score**

Write `check_score(score)` that raises `TypeError` if the score is not an `int`, and `ValueError` if it is not between 0 and 100. Otherwise it returns the score. Call it with `105` inside a `try` block and print the error message.

▼ **Hint**

Use `isinstance(score, int)` for the type check, then a range check.

Discussion

1. Why is a bare `except:` dangerous? Describe a bug it could hide.
2. A user types "twelve" into a grade form. Should the program crash, show a message, or retry? Who decides?
3. When would you rather check with an `if` than with `try`?

Key terms

Exception

An error signal raised when something goes wrong during execution. Unhandled exceptions stop the program.

Traceback

The report Python prints for an unhandled exception, listing the call path and ending with the exception type and message.

try / except

A block that runs code and jumps to a handler if a named exception is raised.

else (try)

A block that runs only when the try block completed without an exception.

finally

A block that always runs after try/except, used for cleanup.

raise

A statement that raises an exception on purpose.

ValueError

Raised when an argument has the right type but an inappropriate value.

TypeError

Raised when an operation is applied to a value of the wrong type.

See all terms in the glossary →

UNIT 12 75 min

Advanced Functions and Comprehensions

Flexible arguments, lambda, map/filter/zip, comprehensions, sorting with key, and recursion.

By the end of this unit you will be able to

- Write a function that accepts any number of positional and keyword arguments
- Use `lambda` for a short, unnamed function
- Transform and select data with `map`, `filter`, and `zip`
- Write list, dictionary, and set comprehensions, and decide when a plain loop is clearer
- Sort with a `key` function
- Write a recursive function with a base case

Flexible arguments: *args and **kwargs

`*args` collects extra positional arguments into a tuple. `**kwargs` collects extra keyword arguments into a dictionary. The names `args` and `kwargs` are conventions; the `*` and `**` are what matter.

flexible.py

```
def total(*numbers):
    return sum(numbers)

def describe(**details):
    for key, value in details.items():
        print(f"{key}: {value}")

print(total(1, 2, 3, 4))
describe(name="Ada", course="Python")
```

lambda: a small unnamed function

`lambda parameters: expression` makes a function with no name. It can hold one expression only. Use it when you need a short function to hand to another function, such as a sort key. Name the

function with `def` when it needs more than one line.

lambda.py

```
square = lambda x: x * x
print(square(7))

pairs = [(1, "pear"), (3, "apple"), (2, "fig")]
print(sorted(pairs))                       # sorts by the first item
print(sorted(pairs, key=lambda p: p[1]))   # sorts by the name
```

map, filter, and zip

`map(func, items)` applies a function to each item. `filter(func, items)` keeps items for which the function returns True. `zip(a, b)` pairs items from two sequences. In modern Python, `map`, `filter`, and `zip` return iterators, so wrap them in `list()` to see the results.

map_filter_zip.py

```
prices = [2.5, 4.0, 1.25]
print(list(map(lambda p: p * 1.07, prices)))
print(list(filter(lambda p: p > 2, prices)))

names = ["Ana", "Ben"]
scores = [91, 84]
print(list(zip(names, scores)))
for name, score in zip(names, scores):
    print(name, score)
```

Comprehensions

A **comprehension** builds a collection in one expression. The pattern is `[expression for item in sequence if condition]`. Use the same idea with `{}` for a dictionary or set.

comprehensions.py

```
nums = range(1, 11)

squares = [n * n for n in nums]
evens = [n for n in nums if n % 2 == 0]
labels = ["even" if n % 2 == 0 else "odd" for n in range(4)]
lengths = {word: len(word) for word in ["cat", "horse"]}
unique_lengths = {len(word) for word in ["cat", "dog", "horse"]}

print(squares)
print(evens)
print(labels)
print(lengths)
print(unique_lengths)
```

> **Tip**
> If a comprehension needs more than one loop or a long condition, write a normal loop. Clear code beats short code. Your reader is the priority.

A **generator expression** uses parentheses instead of brackets. It produces values one at a time, which saves memory on large data: `sum(n * n for n in range(1000))`.

Sorting with key

`sorted()` and `list.sort()` accept a `key` function. Python sorts by the value that function returns, not by the item itself. Add `reverse=True` for descending order.

sort_key.py

```
students = [
    {"name": "Ana", "score": 88},
    {"name": "Ben", "score": 95},
    {"name": "Cy", "score": 72}
]
ranked = sorted(students, key=lambda s: s["score"], reverse=True)
for s in ranked:
    print(s["name"], s["score"])
```

Recursion

A **recursive** function calls itself on a smaller version of the problem. Every recursive function needs a **base case** that stops the calls, and each call must move toward it. Without a base case you get a `RecursionError`.

Trace: factorial(3) TRACE

```
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

print(factorial(3))
```

Step	Line run	Variables after the line	Output so far
1	1	`factorial = function`	—
2	2	`factorial = function, n = 3`	—
3	4	`factorial = function, n = 3`	—
4	2	`factorial = function, n = 2`	—
5	4	`factorial = function, n = 2`	—
6	2	`factorial = function, n = 1`	—
7	3	`factorial = function, n = 1`	—
8	4	`factorial = function, n = 2`	—
9	4	`factorial = function, n = 3`	—
10	6	`factorial = function`	`6`

Predict the output THINK FIRST

Watch for lazy iterators and for lambdas that return values.

```
def f(*args):
    return len(args)

print(f(1, 2, 3))
```

Answer: `3`

```
print(list(map(lambda x: x + 1, [1, 2])))
```

Answer: `[2, 3]`

```
print(list(filter(lambda x: x > 2, [1, 2, 3, 4])))
```

Answer: `[3, 4]`

```
print(list(zip([1, 2], "ab")))
```

Answer: `[(1, 'a'), (2, 'b')]`

```
print([n for n in range(6) if n % 3 == 0])
```

Answer: `[0, 3]`

```
print(sorted(["bb", "a", "ccc"], key=len))
```

Answer: `['a', 'bb', 'ccc']`

```
print(sum(n for n in range(4)))
```

Answer: `6`

Practice

▼ Exercise 1 — Average of anything

Write `average(*values)` that returns the average of any number of arguments. Test it with `average(2, 4, 9)`.

▼ Exercise 2 — Recursive sum

Write `total(items)` that returns the sum of a list using recursion only (no `sum()`, no loops). Start with the base case of an empty list.

▼ Hint

The sum of a list is its first item plus the sum of the rest: `items[1:]`.

Discussion

1. When is a comprehension easier to read than a loop, and when is it harder?
2. Why does `map` return an iterator instead of a list? What does that change for large files?
3. A recursive solution and a loop solution can compute the same answer. What does each one make easier to see?

Key terms

***args**

A parameter that collects extra positional arguments into a tuple.

****kwargs**

A parameter that collects extra keyword arguments into a dictionary.

lambda

A one-expression, unnamed function, written `lambda x: x * 2`.

map

A built-in that applies a function to each item of a sequence, returning an iterator.

filter

A built-in that keeps the items for which a function returns True.

zip

A built-in that pairs up items from several sequences, stopping at the shortest.

Comprehension

A compact expression that builds a list, dictionary, or set from a loop and an optional condition.

Generator expression

A comprehension in parentheses that produces values lazily, one at a time.

Iterator

An object that produces items one at a time and can be looped over once.

Recursion

A function that calls itself on a smaller problem until it reaches a base case.

Base case

The condition that stops a recursive function from calling itself again.

See all terms in the glossary →

UNIT 13 75 min

Files, CSV, and JSON

Read and write text files safely, then load and save the two formats that real data uses: CSV and JSON.

By the end of this unit you will be able to

- Open, read, and write text files with `with open(...)`
- Choose the right file mode: `r`, `w`, `a`
- Read a CSV file row by row with the `csv` module
- Convert between Python data and JSON with `json`
- Explain why file paths and encodings can break a program, and how to avoid it

Why files

Variables disappear when a program ends. Files keep data between runs. Almost every real data task starts with reading a file, and most results end up written to one.

Opening files with with

`open(path, mode)` returns a file object. Use it inside a `with` block. The file is closed automatically when the block ends, even if an error happens inside it.

Mode	Meaning	If the file is missing
`"r"`	Read text	Error: `FileNotFoundError`
`"w"`	Write text; replaces the old contents	Creates the file
`"a"`	Append to the end	Creates the file

write_read.py

```
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("First line\n")
    f.write("Second line\n")

with open("notes.txt", "r", encoding="utf-8") as f:
    text = f.read()
print(text)
```

> **Note**
> In the browser sandbox, files live in a temporary in-memory folder. They last until the page is reloaded. On your computer they are saved to disk.

Reading line by line

A file object can be looped over. Python reads one line at a time, so this works even for files too large to fit in memory. Each line keeps its newline character; `.strip()` removes it.

lines.py

```
with open("notes.txt", "a", encoding="utf-8") as f:
    f.write("Third line\n")

with open("notes.txt", encoding="utf-8") as f:
    for number, line in enumerate(f, start=1):
        print(number, line.strip())
```

Paths

A path that does not start with a slash is relative to the folder the program runs from. Print `os.getcwd()` if you are unsure. `pathlib` (Unit 15) gives a cleaner way to build paths that work on Windows, macOS, and Linux.

> **Watch out**
> Windows paths contain backslashes. Use a raw string (`r"C:\data\grades.csv"`) or forward slashes (`"C:/data/grades.csv"`).

CSV: rows of comma-separated values

CSV is the most common export format for spreadsheets and databases. The `csv` module handles quoting and commas inside values for you. Always pass `newline=""` when opening a CSV file for writing.

csv_demo.py

```
import csv
import io

# A small CSV held in memory so this runs anywhere.
data = io.StringIO("name,score\nAna,91\nBen,84\n")

reader = csv.DictReader(data)
for row in reader:
    print(row["name"], int(row["score"]) + 1)
```

With `DictReader`, each row is a dictionary keyed by the header. Values always arrive as text, so convert numbers with `int()` or `float()`.

csv_write.py

```
import csv

rows = [{"item": "pencil", "qty": 4}, {"item": "eraser", "qty": 2}]
with open("stock.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["item", "qty"])
    writer.writeheader()
    writer.writerows(rows)

with open("stock.csv", encoding="utf-8") as f:
    print(f.read())
```

JSON: nested data

JSON stores dictionaries, lists, strings, numbers, booleans, and null. Web APIs and configuration files use it. `json.dumps()` turns Python data into a JSON string, and `json.loads()` turns a string back into Python. The `json.dump()` and `json.load()` versions read and write files directly.

json_demo.py

```
import json

student = {"name": "Maya", "grades": [91, 88], "active": True, "advisor": None}

text = json.dumps(student, indent=2)
print(text)

back = json.loads(text)
print(back["grades"][1])
```

Python	JSON
`dict`	object `{ }`
`list` / `tuple`	array `[ ]`
`str`	string
`int` / `float`	number
`True` / `False`	`true` / `false`
`None`	`null`

> **Tip**
> Notice that JSON uses lowercase `true` and `null`. Converting back to Python turns them into `True` and `None`.

Handling file errors

Files go missing and paths change. Combine what you learned in Unit 11 with `open()`.

safe_read.py

```
def read_or_default(path, default=""):
    try:
        with open(path, encoding="utf-8") as f:
            return f.read()
    except FileNotFoundError:
        return default

print(repr(read_or_default("does_not_exist.txt", "no file")))
```

Predict the output THINK FIRST

Think about what each file operation leaves behind.

```
with open("a.txt", "w") as f:
    f.write("hi")
with open("a.txt") as f:
    print(f.read())
```

Answer: `hi`

```
with open("a.txt", "a") as f:
    f.write("!")
with open("a.txt") as f:
    print(f.read())
```

Answer: `hi!`
Append adds to the end. It does not replace the file.

```
import json
print(json.dumps({"ok": True, "x": None}))
```

Answer: `{"ok": true, "x": null}`

```
import json
print(json.loads("[1, 2, 3]")[2])
```

Answer: `3`

Practice

▼ **Exercise 1 — Count lines**

Write a program that writes three lines of text to `log.txt`, then reopens the file and prints how many lines it has.

▼ Exercise 2 — Average from CSV

Using the in-memory CSV below, print the average score as a float.

```
name,score
Ana,91
Ben,84
Cy,77
```

▼ Hint

Collect the scores into a list inside the loop, converting each one with `float()`.

Discussion

1. Why does `with open(...)` matter? What could go wrong with a plain `open` and no `close`?
2. A CSV export opens correctly in a spreadsheet but breaks in your code. What are three things you would check first?
3. JSON and CSV both store data as text. What does each handle well, and what does each handle badly?

Key terms

File object

The object returned by `open()` that provides read and write methods for a file.

with statement

A block that acquires a resource and guarantees it is released when the block ends.

Mode

The letter(s) passed to `open()` that set reading, writing, or appending, such as `"r"`, `"w"`, or `"a"`.

Encoding

The rule for turning characters into bytes, such as UTF-8. Mismatched encodings garble text.

Relative path

A file path measured from the folder the program runs in, rather than from the root of the disk.

CSV

Comma-separated values: plain text with one record per line and commas between fields.

JSON

JavaScript Object Notation: a text format for nested data made of objects, arrays, strings, numbers, booleans, and null.

Serialize

Convert data in memory into a format that can be stored or sent, such as a JSON string.

See all terms in the glossary →

UNIT 14 80 min

Classes and Objects

Bundle data and behavior together: define classes, create objects, write methods, and use inheritance.

By the end of this unit you will be able to

- Define a class and create objects (instances) from it
- Write `__init__` to set up each object, and explain `self`
- Write methods that read and change an object's attributes
- Tell class attributes from instance attributes
- Create a subclass that reuses and extends a parent class with `super()`
- Use a dataclass for a simple record

Why classes

So far, data and functions have been separate: a dictionary holds a student's name and grades, and a function computes the average from it. A **class** combines the two. It defines a kind of object, with data (**attributes**) and behavior (**methods**) kept together. Every value you have used, such as a list or a string, is already an object of some class.

A first class

student.py

```
class Student:
    def __init__(self, name):
        self.name = name
        self.grades = []

    def add_grade(self, grade):
        self.grades.append(grade)

    def average(self):
        return sum(self.grades) / len(self.grades)

maya = Student("Maya")
maya.add_grade(91)
maya.add_grade(88)
print(maya.name, maya.average())
```

- `class Student:` defines the blueprint. By convention, class names are CapitalizedWords.
- `__init__` runs once when you create an object. It sets up the starting attributes.
- `self` is the object the method was called on. It is always the first parameter of a method, and Python fills it in for you.
- `Student("Maya")` creates an **instance**. Each instance has its own separate data.

Trace: two objects, separate data TRACE

```
class Dog:
    def __init__(self, name):
        self.name = name

    def bark(self):
        return self.name + " says woof"

a = Dog("Rex")
b = Dog("Milo")
print(a.bark())
print(b.name)
```

Step	Line run	Variables after the line	Output so far
1	5	`Dog = class`	`—`
2	8	`Dog = class, a = Dog(name="Rex")`	`—`
3	9	`Dog = class, a = Dog(name="Rex"), b = Dog(name="Milo")`	`—`
4	10	`Dog = class, a = Dog(name="Rex"), b = Dog(name="Milo")`	`Rex says woof`
5	11	`Dog = class, a = Dog(name="Rex"), b = Dog(name="Milo")`	`Rex says woof | Milo`

Class attributes and instance attributes

An attribute set on `self` belongs to one object. An attribute set directly in the class body belongs to the class, and every object shares it. Use class attributes for constants or counters that describe the whole group.

class_attr.py

```
class Course:
    school = "Denmark High School"   # shared by every Course

    def __init__(self, title):
        self.title = title           # one per object

python = Course("Python")
ai = Course("AI")
print(python.school, "|", ai.title)
```

Special methods: __str__

Methods named with double underscores on both sides are **special methods**. Python calls them automatically. `__str__` controls what `print()` shows for your object.

str_method.py

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return f"({self.x}, {self.y})"

p = Point(3, 4)
print(p)
print(str(p))
```

Inheritance

A **subclass** inherits everything from a parent class and can add or replace methods. Write `class Child(Parent):`. Call `super().__init__()` to run the parent's setup before adding your own.

inheritance.py

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return "..."

class Cat(Animal):
    def speak(self):
        return f"{self.name} says meow"

class Cow(Animal):
    def speak(self):
        return f"{self.name} says moo"

for pet in [Cat("Ivy"), Cow("Bess")]:
    print(pet.speak())
```

Both subclasses replace `speak`. This is **polymorphism**: the loop does not care which kind of animal it has, because each one knows how to speak.

Dataclasses: records without boilerplate

When a class is only a bundle of fields, the `dataclasses` module writes `__init__` and `__repr__` for you. `@dataclass` is a **decorator**: a line above the class that changes it.

dataclass.py

```
from dataclasses import dataclass

@dataclass
class Book:
    title: str
    pages: int

b = Book("Dune", 412)
print(b)
print(b.pages > 300)
```

Tip

Use a plain class when behavior matters most. Use a dataclass when the object is mostly data, such as a row of a table.

Predict the output THINK FIRST

Trace what each object holds and what each method returns.

```
class C:
    def __init__(self, n):
        self.n = n

print(C(5).n)
```

Answer: `5`

```
class C:
    count = 0
    def __init__(self):
        C.count += 1

C()
C()
print(C.count)
```

Answer: `2`
The shared class attribute counts every object created.

```
class A:
    def hi(self):
        return "A"
class B(A):
    pass
print(B().hi())
```

Answer: `A`
B has no hi of its own, so it inherits A's.

```
class P:
    def __init__(self, x):
        self.x = x
    def __str__(self):
        return f"P{self.x}"
print(P(7))
```

Answer: `P7`

Practice

▼ Exercise 1 — Bank account

Write a class `Account` with an opening balance set in `__init__`, a `deposit(amount)` method, and a `withdraw(amount)` method that refuses (returns `False`) when the balance is too low. Deposit 100, withdraw 30, then try to withdraw 500 and print the final balance.

▼ Hint

Check `amount <= self.balance` before subtracting.

▼ Exercise 2 — Shapes

Create a class `Rectangle` with `width` and `height`, a method `area()`, and a subclass `Square` whose constructor takes one size. Print the area of a `Square(4)`.

Discussion

1. What is the difference between a class and an object? Use a real-world example that is not a computer.
2. When a method uses data from many different parts of a program, is it a sign that the data should belong to a class? Why or why not?
3. Inheritance can make code shorter or harder to follow. When would you avoid it?

Key terms

Class

A blueprint that defines the attributes and methods shared by a kind of object.

Object / instance

A specific value created from a class. Each instance has its own attributes.

Attribute

A variable that belongs to an object, read with a dot, such as `maya.name`.

Method

A function defined inside a class. It receives the object as `self`.

self

The conventional name of the first parameter of a method, referring to the object the method was called on.

__init__

The special method that sets up a new object when it is created.

__str__

The special method that returns the text shown by `print()` and `str()`.

Inheritance

A class that takes over the attributes and methods of a parent class, written `class Child(Parent):`.

super()

A way to call a method from the parent class, such as the parent's `__init__`.

Polymorphism

Using one interface on different kinds of objects, where each responds in its own way.

Dataclass

A class decorated with `@dataclass` that generates the constructor and printable form from its field annotations.

Decorator

A line starting with `@` placed above a function or class that wraps or modifies it.

See all terms in the glossary →

UNIT 15 75 min

The Standard Library Toolkit

Counter, defaultdict, dates, paths, regular expressions, and itertools: tools you will reach for in every data project.

By the end of this unit you will be able to

- Count items with `collections.Counter` and group with `defaultdict`
- Do date arithmetic with `datetime` and format dates as text
- Build file paths with `pathlib`
- Search and extract text with regular expressions (`re`)
- Combine items with `itertools` product and combination helpers

Why the standard library matters

Before installing anything, check the standard library. Most small tasks have a built-in answer, and the built-in version is documented, tested, and available on every machine. Unit 10 covered how to import and read docs; this unit covers the tools you will use most.

collections.Counter and defaultdict

`Counter` counts hashable items. It replaces the counting loop from Unit 9. `defaultdict(list)` gives every new key an empty list automatically, so you can append without checking first.

collections_demo.py

```
from collections import Counter, defaultdict

words = "the cat and the hat and the bat".split()
counts = Counter(words)
print(counts.most_common(2))

by_length = defaultdict(list)
for w in words:
    by_length[len(w)].append(w)
print(dict(by_length))
```

Note

`most_common(n)` returns the n most frequent items with their counts. When two items tie, the one that appeared first comes first.

datetime: dates and times

`date` holds a calendar day, `datetime` holds a day and a time, and `timedelta` represents a difference. Subtracting two dates gives a `timedelta`. `strftime` formats a date as text, and `strptime` parses text into a date.

dates.py

```
from datetime import date, timedelta, datetime

start = date(2026, 8, 26)
due = start + timedelta(days=21)
print(due)
print((due - start).days)
print(due.strftime("%B %d, %Y"))

parsed = datetime.strptime("2026-09-01 14:30", "%Y-%m-%d %H:%M")
print(parsed.hour)
```

Code	Meaning	Example output
`%Y`	four-digit year	2026
`%m` / `%d`	month / day, zero-padded	09 / 01
`%B`	full month name	September
`%H:%M`	hour (24-hour) and minute	14:30

> **Watch out**
> Naive `datetime` objects have no time zone. Mixing them with timezone-aware ones raises an error. Store times in one zone (often UTC) and convert only for display.

pathlib: paths that work everywhere

`Path` objects join folders with `/`, which works on Windows, macOS, and Linux. They also know how to check whether a file exists and how to read it.

pathlib_demo.py

```
from pathlib import Path

folder = Path("data")
grades = folder / "grades.csv"
print(grades)
print(grades.suffix)
print(grades.stem)
print(grades.parent)
```

Paths are only descriptions. Use `grades.exists()` to check, and `grades.read_text()` or `grades.write_text()` to work with the contents.

re: regular expressions

A regular expression describes a pattern of text. The `re` module finds, checks, and replaces text that matches. Write patterns as raw strings (`r"..."`) so backslashes reach the regex engine unchanged.

Pattern	Matches
`\d`	one digit (0–9)
`\w`	a letter, digit, or underscore
`\s`	whitespace
`.`	any character except newline
`+` / `*`	one or more / zero or more of the item before it
`[a-z]`	any lowercase letter
`^` / `$`	start / end of the text

re_demo.py

```
import re

log = "Order 1042 shipped to zip 30041 on 2026-08-26"

print(re.findall(r"\d+", log))
print(re.search(r"\d{4}-\d{2}-\d{2}", log).group())
print(re.sub(r"\d", "#", "room 204"))

m = re.match(r"(\w+)@(\w+)\.com", "ada@school.com")
print(m.group(1), m.group(2))
```

> **Tip**
> Start with a simple pattern and test it on a few examples. Regular expressions are powerful, and they are also easy to get wrong. Keep them short, and comment what they match.

itertools: combining items

`itertools` has fast tools for combinations. `product` gives every pair, `combinations` gives each unordered group once, and `permutations` gives each ordering.

itertools_demo.py

```
from itertools import product, combinations

sizes = ["S", "M"]
colors = ["red", "blue"]
print(list(product(sizes, colors)))

team = ["Ana", "Ben", "Cy"]
print(list(combinations(team, 2)))
```

Predict the output THINK FIRST

Watch for ties, zero-padding, and what `re` returns when nothing matches.

```
from collections import Counter
print(Counter("aab").most_common(1))
```

Answer: `[('a', 2)]`

```
from datetime import date, timedelta
print((date(2026, 1, 1) + timedelta(days=31)).isoformat())
```

Answer: `2026-02-01`

```
from pathlib import Path
print(Path("a/b/c.txt").suffix)
```

Answer: `.txt`

```
import re
print(re.findall(r"\d+", "a1b22c333"))
```

Answer: `['1', '22', '333']`

```
import re
print(re.search(r"z", "abc"))
```

Answer: `None`
`search` returns None when nothing matches.

```
from itertools import combinations
print(len(list(combinations([1, 2, 3, 4], 2))))
```

Answer: `6`

Practice

▼ **Exercise 1 — Most common letter**

Using `Counter`, print the most common letter in `"mississippi"` and how many times it appears.

▼ **Exercise 2 — Extract emails**

Given `text = "Contact ada@school.org or ben@example.com today."`, use `re.findall` to print a list of the two addresses. A simple pattern is `r"[\w.]+@[\w.]+"`.

Discussion

1. Which standard-library tool from this unit would have saved you the most time in a past project?
2. Regular expressions are hard to read. When is that cost worth paying, and when should you use plain string methods instead?
3. Why does pathlib use `/` to join paths? What would a hand-written path join need to handle?

Key terms

Counter

A dictionary subclass from `collections` that counts how often each item appears.

defaultdict

A dictionary that creates a default value (such as an empty list) for any missing key.

timedelta

A duration: the difference between two dates or times.

strftime / strptime

Methods that format a date as text (`strftime`) or parse text into a date (`strptime`).

Path

An object from `pathlib` that represents a file or folder path and offers methods to read, write, and inspect it.

Regular expression

A pattern that describes a set of strings, used to search, match, and replace text.

Raw string

A string literal prefixed with `r` that treats backslashes literally, commonly used for regex patterns.

itertools

A standard-library module of fast, memory-efficient tools for looping and combining items.

See all terms in the glossary →

UNIT 16 90 min

NumPy: Arrays and Fast Math

Arrays with a fixed type and shape, math without loops, indexing and masks, broadcasting, and the matrix product.

By the end of this unit you will be able to

- Create arrays and explain how they differ from Python lists
- Read an array's `shape`, `dtype`, and `ndim`
- Use vectorized math instead of loops, and explain why it is faster
- Index, slice, and select with boolean masks in 1-D and 2-D arrays
- Aggregate along an axis and predict the shape of the result
- Use broadcasting and the `@` matrix product

Why NumPy

NumPy stores numbers in a compact block of memory, all with the same type, and runs its math in optimized code. An operation on a whole array happens in one step, with no Python loop. This is the foundation for pandas, scikit-learn, and PyTorch: all of them use NumPy-style arrays underneath.

import_numpy.py

```
import numpy as np

a = np.array([1, 2, 3, 4])
print(a)
print(a * 10)
print(a.shape, a.dtype)
```

Note

The convention is `import numpy as np`. Nearly every tutorial and codebase uses `np`.

Arrays versus lists

	List	NumPy array
Element types	Can mix types	One type for all items (`dtype`)
`[1, 2] * 2`	`[1, 2, 1, 2]` (repeats)	`[2 4]` (multiplies each item)
`+` between two	Joins them	Adds item by item
Math on every item	Needs a loop or comprehension	Written directly
Speed on large data	Slower	Much faster

> **Watch out**
> `[1, 2] * 2` repeats the list. `np.array([1, 2]) * 2` multiplies each number. Mixing these up is the most common NumPy beginner error.

Creating arrays

creating.py

```
import numpy as np

print(np.zeros(3))
print(np.ones((2, 3)))
print(np.arange(0, 10, 2))
print(np.linspace(0, 1, 5))
print(np.eye(2))
```

`np.arange(start, stop, step)` works like `range` but returns an array. `np.linspace(start, stop, count)` gives evenly spaced values and includes `stop`. The shape is a tuple: `(2, 3)` means two rows, three columns.

Shape, dtype, and reshape

shape.py

```
import numpy as np

a = np.arange(6)
print(a.shape)
grid = a.reshape(2, 3)
print(grid)
print(grid.shape, grid.ndim)
print(np.array([1.5, 2]).dtype)
```

`reshape` changes the shape without changing the data. The new shape must hold the same number of items. Use `-1` to let NumPy calculate one dimension for you: `a.reshape(-1, 2)` gives three rows here.

Indexing and slicing

For a 2-D array, `grid[row, col]` picks one element. A colon selects a whole range. `grid[:, 1]` means "all rows, column 1".

indexing.py

```
import numpy as np

grid = np.arange(1, 13).reshape(3, 4)
print(grid)
print(grid[0, 2])
print(grid[1])
print(grid[:, 1])
print(grid[1:, :2])
```

Boolean masks

Comparing an array with a value gives a **boolean mask**: an array of True and False with the same shape. Use the mask inside square brackets to keep only the True positions. This is how you filter

data without a loop.

masks.py

```
import numpy as np

scores = np.array([72, 95, 88, 61, 90])
passing = scores >= 75
print(passing)
print(scores[passing])
print(np.count_nonzero(passing))
scores[scores < 65] = 65
print(scores)
```

Tip

Combine masks with `&` (and), `|` (or), and `~` (not). Put each comparison in parentheses: `(a > 1) & (a < 5)`. Plain `and` does not work on arrays.

Vectorized math and aggregation

Operations like `+`, `*`, `np.sqrt`, and `np.exp` apply to every element. Aggregations like `sum`, `mean`, `std`, `min`, and `max` reduce an array. With `axis=0` they reduce down the rows (one result per column). With `axis=1` they reduce across the columns (one result per row).

aggregate.py

```
import numpy as np

grid = np.array([[1, 2, 3],
                 [4, 5, 6]])
print(grid.sum())
print(grid.sum(axis=0))
print(grid.sum(axis=1))
print(grid.mean(axis=1))
print(np.sqrt(np.array([4, 9, 16])))
```

Broadcasting

When two arrays have different shapes, NumPy **broadcasts** the smaller one: it stretches it across the larger one without copying data. A single number, or a row that matches the last dimension, can be combined with a whole table.

broadcast.py

```
import numpy as np

table = np.array([[10, 20, 30],
                  [40, 50, 60]])
bonus = np.array([1, 2, 3])
print(table + bonus)
print(table * 0.5)
```

Trace: what broadcasting does TRACE

```
import numpy as np

table = np.array([[10, 20], [30, 40]])
bonus = np.array([1, 2])
result = table + bonus
print(result)
```

Step	Line run	Variables after the line	Output so far
1	3	`np = module, table = 2 x 2 array`	—
2	4	`np = module, table = 2 x 2 array, bonus = shape (2,)`	—
3	5	`np = module, table = 2 x 2 array, bonus = shape (2,), result = [[11 22] [31 42]]`	—
4	6	`np = module, table = 2 x 2 array, bonus = shape (2,), result = [[11 22] [31 42]]`	`[[11 22] [31 42]]`

Matrix product

The `@` operator performs matrix multiplication. The inner sizes must match: a (2, 3) array times a (3, 4) array gives a (2, 4) array. `*` is different: it multiplies element by element.

matmul.py

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[0, 1], [1, 0]])
print(A * B)
print(A @ B)
```

Random numbers with a seed

For reproducible results, create a generator with a seed. The same seed gives the same numbers every run, which matters for tests and for teaching.

rng.py

```
import numpy as np

rng = np.random.default_rng(42)
print(rng.integers(1, 7, size=5))
print(rng.random(2).round(3))
```

Predict the output THINK FIRST

Pay attention to shape, and to `*` versus `@`.

```
import numpy as np
print(np.array([1, 2]) * 2)
```

Answer: `[2 4]`

```
import numpy as np
print(np.zeros((2, 3)).shape)
```

Answer: `(2, 3)`

```
import numpy as np
print(np.arange(5)[::2])
```

Answer: `[0 2 4]`

```
import numpy as np
g = np.arange(6).reshape(2, 3)
print(g.sum(axis=0))
```

Answer: `[3 5 7]`
Columns: 0+3, 1+4, 2+5.

```
import numpy as np
print(np.array([1, 5, 9]) > 4)
```

Answer: `[False True True]`

```
import numpy as np
print(np.arange(6).reshape(2, 3).reshape(-1).shape)
```

Answer: `(6,)`

Practice

▼ **Exercise 1 — Temperatures**

Given `celsius = np.array([0, 20, 37, 100])`, compute an array of Fahrenheit values with one expression, and print the values that are above 80°F using a mask.

▼ **Exercise 2 — Column means**

Create a 4-by-3 array with `np.arange(12).reshape(4, 3)`. Print the mean of each column, then the total of all values.

Discussion

1. A list comprehension and a NumPy expression can give the same answer. What does each make easier to read, and what does each make easier to speed up?
2. Why does NumPy require one dtype for all items? What would be lost if it allowed mixed types?
3. Broadcasting can silently combine shapes you did not intend. How would you check that a result has the shape you expect?

Key terms

ndarray

NumPy's n-dimensional array: a block of same-typed values with a shape.

shape

A tuple giving the size of each dimension, such as `(2, 3)` for two rows and three columns.

dtype

The data type of every element in an array, such as `int64` or `float64`.

Vectorization

Writing an operation on a whole array at once instead of looping over its elements.

Boolean mask

An array of True and False values, used to select elements that meet a condition.

Axis

The dimension an aggregation runs along. `axis=0` runs down rows; `axis=1` runs across columns.

Broadcasting

The rules NumPy uses to combine arrays of different shapes by stretching the smaller one.

Reshape

Changing an array's shape while keeping the same data and the same number of elements.

Matrix product

The `@` operator, which multiplies two arrays using rows of the first and columns of the second.

See all terms in the glossary →

UNIT 17 90 min

pandas: Tables You Can Question

Load a table, inspect it, select and filter rows, create columns, handle missing values, and summarize by group.

By the end of this unit you will be able to

- Create a DataFrame and explain how it differs from a NumPy array
- Load a CSV with `pd.read_csv` and inspect it with `head`, `info`, and `describe`
- Select columns and rows with `[]`, `.loc`, and `.iloc`
- Filter rows with conditions and create new columns
- Find, count, and fill missing values
- Summarize with `groupby` and `value_counts`

What pandas adds

A **DataFrame** is a table: rows and columns, where each column has a name and can have its own type. A **Series** is one column. pandas builds on NumPy, so the speed is similar, and it adds labels, missing-value handling, and the reading and grouping tools that real data needs.

first_frame.py

```
import pandas as pd

df = pd.DataFrame({
    "name": ["Ana", "Ben", "Cy"],
    "score": [91, 84, 77],
    "period": [1, 2, 1]
})
print(df)
print(df["score"])
print(type(df["score"]).__name__)
```

> **Note**
> The convention is `import pandas as pd`.

Reading data

`pd.read_csv` loads a CSV file into a DataFrame. It accepts a file path or a URL. Here it reads from text held in memory, so the example runs in the sandbox. With a real file, pass its path instead.

read_csv.py

```
import io
import pandas as pd

csv_text = """name,period,score,absent
Ana,1,91,0
Ben,2,84,2
Cy,1,77,
Dee,2,95,1
"""
df = pd.read_csv(io.StringIO(csv_text))
print(df)
```

Inspecting a table

Call	What you learn
`df.head(3)`	The first rows, to see the layout
`df.shape`	(rows, columns)
`df.info()`	Column names, types, and how many values are missing
`df.describe()`	Count, mean, spread, and quartiles of the numeric columns
`df.columns`	The column names

inspect.py

```
import io
import pandas as pd

df = pd.read_csv(io.StringIO("name,period,score,absent\nAna,1,91,0\nBen,2,84,2\
print(df.shape)
print(df.columns.tolist())
print(df.describe())
```

Selecting: columns, then rows

Select columns with a list of names: `df[["name", "score"]]`. For rows, use `.loc` with labels, or `.iloc` with positions. `.iloc[0]` is the first row; `.loc[0]` is the row labeled 0. With the default index these are the same, but they stop matching once you sort or filter.

select.py

```
import pandas as pd

df = pd.DataFrame({"name": ["Ana", "Ben", "Cy"], "score": [91, 84, 77]})
print(df[["name", "score"]])
print(df.iloc[1])
print(df.loc[2, "score"])
print(df.iloc[0:2, 0])
```

Filtering rows

A condition on a column gives a True/False Series. Put it inside `df[...]` to keep the matching rows. Combine conditions with `&` and `|`, and put each one in parentheses.

filter.py

```
import pandas as pd

df = pd.DataFrame({"name": ["Ana", "Ben", "Cy", "Dee"],
                   "score": [91, 84, 77, 95],
                   "period": [1, 2, 1, 2]})
high = df[df["score"] >= 90]
print(high)
print(df[(df["period"] == 1) & (df["score"] < 90)])
```

New columns and sorting

columns_sort.py

```
import pandas as pd

df = pd.DataFrame({"name": ["Ana", "Ben", "Cy"], "score": [91, 84, 77]})
df["passed"] = df["score"] >= 80
df["curve"] = df["score"] + 5
print(df.sort_values("score", ascending=False))
```

Arithmetic on a column applies to every row, the same way it does in NumPy. Methods like `sort_values` return a new DataFrame, so assign the result if you want to keep it.

Missing values

Missing values appear as `NaN` (not a number). Count them with `isna().sum()`. Remove rows with `dropna()`, or fill them with `fillna()`. Decide which one before you summarize: a mean that silently skips missing values can hide a problem.

missing.py

```
import io
import pandas as pd

df = pd.read_csv(io.StringIO("name,absent\nAna,0\nBen,2\nCy,\n"))
print(df.isna().sum())
print(df.dropna())
print(df.fillna(0))
```

Watch out

An empty cell becomes `NaN`, which makes an integer column into a float column. That is normal, but check `info()` when a count column looks wrong.

groupby: summaries by category

`groupby(column)` splits the table into one group per value, then applies a summary to each group. The pattern is `df.groupby("period")["score"].mean()`: group by period, then take the mean score in each.

groupby.py

```
import pandas as pd

df = pd.DataFrame({
    "name": ["Ana", "Ben", "Cy", "Dee", "Eli"],
    "period": [1, 2, 1, 2, 1],
    "score": [91, 84, 77, 95, 88]
})
print(df.groupby("period")["score"].mean())
print(df.groupby("period")["score"].agg(["count", "min", "max"]))
print(df["period"].value_counts())
```

Tip

Use `agg` when you want several summaries at once. Use `value_counts()` to count categories in one column.

Predict the output THINK FIRST

Think about the shape and index of each result before you run it.

```
import pandas as pd
print(pd.DataFrame({"a": [1, 2]}).shape)
```

Answer: `(2, 1)`

```
import pandas as pd
df = pd.DataFrame({"x": [5, 6, 7]})
print(df[df["x"] > 5]["x"].tolist())
```

Answer: `[6, 7]`

```
import pandas as pd
df = pd.DataFrame({"x": [5, 6, 7]})
print(df.iloc[-1, 0])
```

Answer: `7`

```
import pandas as pd
print(pd.Series([1, None, 3]).isna().sum())
```

Answer: `1`

```
import pandas as pd
df = pd.DataFrame({"g": ["a", "b", "a"], "v": [1, 2, 3]})
print(df.groupby("g")["v"].sum().tolist())
```

Answer: `[4, 2]`

Practice

▼ **Exercise 1 — Honor roll**

Build a DataFrame with the names `Ana, Ben, Cy, Dee` and scores `91, 84, 77, 95`. Add a column `honors` that is True when the score is 90 or more. Print only the rows where `honors` is True.

▼ **Exercise 2 — Period averages**

Using the table from the groupby example, print the average score for each period, rounded to one decimal place with `.round(1)`.

Discussion

1. Why do you think pandas keeps the original index after filtering rows? When does that help, and when does it cause bugs?
2. A column of ages has one blank cell. Options: drop the row, fill with 0, fill with the average. Which would you choose for a survey, and why?
3. What question would you ask of a grade table before you trust a `groupby` average?

Key terms

DataFrame

A two-dimensional labeled table in pandas, with named columns that can each have a different type.

Series

A one-dimensional labeled array, such as a single column of a DataFrame.

Index

The labels for the rows of a DataFrame or Series. The default is 0, 1, 2, and so on.

read_csv

The pandas function that loads a CSV file (or text) into a DataFrame.

loc / iloc

`loc` selects by label; `iloc` selects by integer position.

NaN

Not a Number: pandas' marker for a missing value in a numeric column.

groupby

A method that splits rows into groups by the values of one or more columns, so you can summarize each group.

value_counts

A method that counts how often each distinct value appears in a column.

See all terms in the glossary →

UNIT 18 75 min

Visualizing Data with matplotlib

Choose the right chart, build line, bar, scatter, and histogram plots, label them honestly, and save them.

By the end of this unit you will be able to

- Choose a chart type that matches the question: comparison, trend, relationship, or distribution
- Draw a plot with `matplotlib.pyplot` and add a title, axis labels, and a legend
- Build several plots in one figure with `subplots`
- Plot directly from a pandas DataFrame
- Save a figure to a file and explain why axis choices can mislead

Pick the chart for the question

A chart answers one question. Start with the question, then choose the chart. The most common mistake is choosing a chart because it looks impressive rather than because it answers the question.

Question	Chart	Function
How do groups compare?	Bar chart	`plt.bar`
How does something change over time?	Line chart	`plt.plot`
Are two numbers related?	Scatter plot	`plt.scatter`
How are values spread out?	Histogram	`plt.hist`

Note

The plotting code in this unit is not run in the browser sandbox, because a sandbox cannot display a figure. Run it in Google Colab (free), in a Jupyter notebook, or as a script on your computer. Each example has been run with matplotlib installed.

Your first plot

first_plot.py

```
import matplotlib.pyplot as plt

months = ["Aug", "Sep", "Oct", "Nov"]
hours = [4, 6, 5, 7]

plt.plot(months, hours, marker="o")
plt.title("Study hours per month")
plt.xlabel("Month")
plt.ylabel("Hours")
plt.show()
```

Every chart needs a title, labeled axes with units, and a legend if it has more than one series. `plt.show()` opens the window. In a notebook, the figure appears below the cell.

Bar charts for comparison

bars.py

```
import matplotlib.pyplot as plt

periods = ["1", "2", "3", "4"]
averages = [86.5, 82.0, 90.2, 78.4]

plt.bar(periods, averages, color="#3b5bdb")
plt.title("Average score by period")
plt.xlabel("Period")
plt.ylabel("Average score (out of 100)")
plt.ylim(0, 100)
plt.show()
```

Watch out

Bar charts should start at zero. A bar that starts at 80 makes a 2-point difference look like a 50% difference. That is a misleading chart, even if every number is correct. Line charts and scatter plots are allowed to use a narrower range.

Scatter plots for relationships

scatter.py

```
import matplotlib.pyplot as plt

hours_studied = [1, 2, 3, 4, 5, 6, 7, 8]
test_scores = [55, 60, 66, 70, 74, 79, 85, 88]

plt.scatter(hours_studied, test_scores)
plt.title("Study time and test score")
plt.xlabel("Hours studied")
plt.ylabel("Test score")
plt.show()
```

A scatter plot shows whether two numbers move together. It does not prove that one causes the other. Say "is associated with" unless you have an experiment.

Histograms for distributions

hist.py

```
import matplotlib.pyplot as plt

scores = [55, 62, 68, 70, 71, 73, 75, 78, 79, 80, 82, 85, 88, 91, 95]

plt.hist(scores, bins=5, edgecolor="white")
plt.title("Distribution of test scores")
plt.xlabel("Score")
plt.ylabel("Number of students")
plt.show()
```

The number of bins changes the picture. Too few hides detail, and too many makes noise. Try a few values before you decide what the data shows.

Several plots in one figure

`plt.subplots(rows, cols)` returns a figure and a grid of axes. Draw on each axis with its own method: `ax.plot`, `ax.bar`, and so on. Use `fig.tight_layout()` to stop labels from overlapping.

subplots.py

```
import matplotlib.pyplot as plt

hours = [1, 2, 3, 4, 5]
scores = [55, 62, 66, 73, 79]

fig, (left, right) = plt.subplots(1, 2, figsize=(9, 3.5))
left.plot(hours, scores, marker="o")
left.set_title("Trend")
right.scatter(hours, scores)
right.set_title("Relationship")
fig.tight_layout()
plt.show()
```

Plotting a DataFrame

pandas DataFrames and Series have a `.plot` method that calls matplotlib for you. It is a fast way to look at data, but name your axes explicitly before sharing a chart.

df_plot.py

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({"period": [1, 2, 3], "score": [86, 82, 90]})
ax = df.plot(x="period", y="score", kind="bar", legend=False)
ax.set_ylabel("Average score")
plt.tight_layout()
plt.show()
```

Saving a figure

`plt.savefig("chart.png", dpi=150)` writes the figure to a file. Call it *before* `plt.show()`, because `show` can clear the figure. Use PNG for slides and SVG or PDF for documents.

Judgment checks

- **Bars start at 80 instead of 0.** Exaggerates small differences, because the eye compares bar heights.
- **A scatter plot shows study hours and scores rising together.** That is an association. Other factors can drive both numbers.
- **A histogram with 2 bins for 40 scores.** Hides the shape, because different parts of the distribution are merged.

Practice

▼ **Exercise 1 — Labeled trend**

Plot the list `[3, 5, 4, 8, 7]` against the days `["Mon", "Tue", "Wed", "Thu", "Fri"]` as a line with markers. Add a title, both axis labels, and save it to `trend.png`.

Discussion

1. Find a chart in a newspaper or on social media. What question does it answer? Would a different chart answer it better?
2. When is a truncated axis acceptable? Give one example where it is and one where it is not.
3. What would you need to know before claiming that a scatter plot shows a cause?

Key terms

matplotlib

The standard Python library for drawing charts. Its `pyplot` module is usually imported as `plt`.

Figure

The whole picture that matplotlib draws, which can hold one or more axes.

Axes

A single plotting area inside a figure, with its own x and y axes, title, and data.

Histogram

A chart that groups numeric values into bins and shows how many fall in each bin.

Bin

A range of values grouped together in a histogram.

Scatter plot

A chart of points, one per observation, that shows the relationship between two numbers.

Truncated axis

An axis that does not start at zero, which can exaggerate differences between bars.

subplots

A function that creates a grid of axes in one figure.

See all terms in the glossary →

UNIT 19 90 min

Machine Learning with scikit-learn

The supervised-learning workflow: features and labels, train/test splits, fitting models, measuring accuracy, and avoiding overfitting.

By the end of this unit you will be able to

- Explain features, labels, and the difference between supervised and unsupervised learning
- Split data into training and test sets, and explain why the split must come first
- Fit a model with `fit`, make predictions with `predict`, and measure it with `score`
- Read a confusion matrix and explain what accuracy does and does not tell you
- Recognize overfitting and use a pipeline so preprocessing does not leak test information

What machine learning is

Machine learning finds patterns in data and uses them to make predictions. Instead of writing the rules yourself, you give the algorithm examples and let it estimate the rules. The quality of a model depends mostly on the data and on how honestly you measure it.

- **Features** are the input measurements, such as petal length or hours studied. Usually a table with one row per example.
- **Label** (or target) is the answer you want to predict, such as the flower species.
- **Supervised learning** uses labeled examples. Classification predicts a category; regression predicts a number.
- **Unsupervised learning** finds structure without labels, such as grouping similar customers. It is covered briefly in the capstone.

Load a dataset

scikit-learn ships with small classic datasets, so you can start without downloading anything. The iris dataset has 150 flowers, four measurements each, and three species.

load_iris.py

```
from sklearn.datasets import load_iris

iris = load_iris()
print(iris.feature_names)
print(iris.target_names)
print(iris.data.shape)
print(iris.data[0])
print(iris.target[0])
```

`iris.data` is the feature table (150 rows, 4 columns). `iris.target` holds the label for each row as a number: 0, 1, or 2. `iris.target_names` says what each number means.

Train and test sets

A model must be tested on examples it has never seen. Hold back part of the data as a **test set** before any training. Train on the rest. If you test on the data you trained on, the score measures memory, not learning.

split.py

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.25, random_state=42
)
print(len(X_train), len(X_test))
```

Note

By convention, `X` (capital) holds the features and `y` holds the labels. `random_state=42` makes the split the same every run. The number 42 is arbitrary; any fixed number works.

Fit, predict, score

Every scikit-learn model follows the same pattern. Create it, call `fit(X_train, y_train)` to learn from the training data, call `predict(X_test)` to get predictions, and call `score(X_test, y_test)` to get accuracy. Once you learn this pattern, you can swap models freely.

first_model.py

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.25, random_state=42
)

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

predictions = model.predict(X_test)
print(predictions[:10])
print("accuracy:", round(model.score(X_test, y_test), 3))
```

K-nearest neighbors predicts the majority label among the *k* closest training examples. It is simple to understand, which makes it a good first model.

Accuracy and the confusion matrix

Accuracy is the fraction of correct predictions. It can hide problems: if 95% of emails are not spam, a model that always says "not spam" scores 95%. A **confusion matrix** shows where each kind of mistake happens. Rows are the true labels, columns are the predictions.

confusion.py

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.25, random_state=42
)
model = KNeighborsClassifier(n_neighbors=3).fit(X_train, y_train)
pred = model.predict(X_test)

print(accuracy_score(y_test, pred))
print(confusion_matrix(y_test, pred))
```

Tip

Chaining `.fit(...)` works because `fit` returns the model itself. It is a common pattern in scikit-learn.

Overfitting

Overfitting means the model learned the training examples, including their noise, and does poorly on new data. A model with a very large `k` is too simple; a model with `k=1` can memorize. Compare training and test scores to see the gap. A large gap is a warning.

overfit.py

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.25, random_state=42
)

for k in [1, 3, 5, 15]:
    model = KNeighborsClassifier(n_neighbors=k).fit(X_train, y_train)
    train_acc = model.score(X_train, y_train)
    test_acc = model.score(X_test, y_test)
    print(k, round(train_acc, 2), round(test_acc, 2))
```

Preprocessing with a pipeline

Some models need features on a similar scale. `StandardScaler` rescales each feature to mean 0 and standard deviation 1. The scaler must learn its numbers from the **training** data only. If you scale the whole table first, information from the test set leaks into training, and the score is too optimistic. A `Pipeline` does the steps in the right order for you.

pipeline.py

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.25, random_state=42
)

pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=500))
pipe.fit(X_train, y_train)
print("accuracy:", round(pipe.score(X_test, y_test), 3))
```

Regression: predicting a number

When the label is a number, use a regressor. `LinearRegression` fits a straight line (or plane) through the data. The score for regression is R^2, where 1.0 is perfect and 0 means no better than predicting the average.

regression.py

```
from sklearn.linear_model import LinearRegression

hours = [[1], [2], [3], [4], [5]]
scores = [52, 58, 65, 71, 76]

line = LinearRegression().fit(hours, scores)
print(round(line.coef_[0], 2))
print(round(line.predict([[6]])[0], 1))
```

Predict the output THINK FIRST

Trust the process: predict what the model will do with each change.

```
from sklearn.datasets import load_iris
print(len(load_iris().target))
```

Answer: `150`

```
import numpy as np
print(np.bincount([0, 0, 1, 2]))
```

Answer: `[2 1 1]`
Counts how often each label value appears.

```
from sklearn.linear_model import LinearRegression
m = LinearRegression().fit([[0], [1], [2]], [0, 2, 4])
print(round(m.predict([[10]])[0]))
```

Answer: `20`
The data follows y = 2x exactly, so the fitted line does too.

Practice

▼ **Exercise 1 — Try other k values**

Using the iris split from this unit (`random_state=42` , `test_size=0.25`), find which value of `k` from 1 to 15 gives the highest test accuracy. Print that `k` and its accuracy.

▼ **Hint**

Loop over `range(1, 16)` , keep track of the best score and its k.

▼ **Exercise 2 — Spot the leak**

The code below scales all the data before splitting. Explain in one sentence what is wrong with it, then rewrite it so the scaler is fitted on the training set only.

```
X_scaled = StandardScaler().fit_transform(iris.data)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, iris.target, r
```

Discussion

1. A spam filter is 99% accurate. Why might that still be a bad filter? What would you check next?
2. Why must the test set stay untouched until the very end of a project? What happens to your honest score if you tune the model using it?
3. Which features in the iris data would a person use to tell species apart? Does the model use them the same way?

Key terms

Feature

An input variable used to make a prediction, usually a column in the feature table.

Label / target

The value a supervised model is trained to predict.

Supervised learning

Learning from examples that include the correct label.

Unsupervised learning

Finding structure in data that has no labels, such as clusters.

Train/test split

Dividing data into a set used for fitting and a held-back set used only for evaluation.

Accuracy

The fraction of predictions that match the true labels.

Confusion matrix

A table of true labels against predicted labels, showing which mistakes the model makes.

Overfitting

When a model fits the training data, including its noise, so well that it performs worse on new data.

Data leakage

When information from the test set influences training, making the measured score too optimistic.

Pipeline

A scikit-learn object that chains preprocessing steps and a model so they are fitted and applied in order.

R^2

A regression score where 1.0 is a perfect fit and 0 means no better than predicting the average.

See all terms in the glossary →

UNIT 20 90 min

PyTorch: Tensors, Gradients, and Neural Networks

Tensors, the operations on them, automatic differentiation, and building a small neural network with nn.Module.

By the end of this unit you will be able to

- Create tensors and check their shape, dtype, and device
- Explain how a tensor relates to a NumPy array, and convert between them
- Use `requires_grad` and `backward()` to compute gradients automatically
- Explain what a neuron, a layer, and an activation function do
- Define a small network with `nn.Module` and run a forward pass

Note

Where to run this unit. PyTorch is not available in the browser sandbox. Use Google Colab (free, no installation; set Runtime → Run all) or install PyTorch locally with `pip install torch`. Every code block here was run with PyTorch 2.x on CPU. Output shown in the text may differ slightly in the last decimal place.

Tensors

A **tensor** is an n-dimensional array, like a NumPy array, with two extra abilities: it can live on a GPU, and it can track how it was computed so gradients can be found automatically. Most of the code you write looks like NumPy.

tensors.py

```
import torch

x = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
print(x)
print(x.shape, x.dtype)
print(torch.zeros(2, 3))
print(torch.arange(5))
```

NumPy	PyTorch	Notes
`np.array(...)`	`torch.tensor(...)`	Copies the data
`np.zeros(...)`	`torch.zeros(...)`	Same idea
`a.shape`	`t.shape`	Same idea
`a.dtype`	`t.dtype`	Default float is float32 in PyTorch, float64 in NumPy
`a @ b`	`t @ u`	Matrix product works the same way

Watch out

Tensors and arrays must have compatible dtypes for many operations. A common error is mixing `int64` with `float32`. Convert with `t.float()` when a function asks for floats.

convert.py

```
import numpy as np
import torch

a = np.array([1.5, 2.5])
t = torch.from_numpy(a)        # shares memory with a
back = t.numpy()               # back to NumPy
print(t, back)
```

Operations and devices

Operations work element by element, use broadcasting, and matrix products with `@`, exactly as in NumPy. Tensors can move to a GPU with `.to(device)`. Code that picks the device at the top keeps working on machines without a GPU.

device.py

```
import torch

device = "cuda" if torch.cuda.is_available() else "cpu"
x = torch.ones(2, 3, device=device)
w = torch.full((3, 2), 0.5, device=device)
print(device, (x @ w).shape)
```

Automatic gradients

Training a model means adjusting numbers to reduce an error. The adjustment is the **gradient**: how much the error changes when each number changes. PyTorch computes gradients automatically. Set `requires_grad=True` on the tensors you want gradients for, compute a result, then call `backward()` on it. The gradients appear in `.grad`.

autograd.py

```
import torch

x = torch.tensor(3.0, requires_grad=True)
y = x ** 2 + 2 * x      # y = x^2 + 2x

y.backward()            # compute dy/dx
print(x.grad)           # 2x + 2 at x = 3 is 8
```

Trace: a gradient by hand TRACE

```
x = 3.0
y = x ** 2 + 2 * x
slope = 2 * x + 2
print(slope)
```

Step	Line run	Variables after the line	Output so far
1	1	`x = 3.0`	—
2	2	`x = 3.0, y = 15.0`	—
3	3	`x = 3.0, y = 15.0, slope = 8.0`	—
4	4	`x = 3.0, y = 15.0, slope = 8.0`	`8.0`

Gradients accumulate in `.grad`. Clear them before each new step with `optimizer.zero_grad()` (Unit 21), or the numbers add up across steps.

A neuron and an activation

A **neuron** multiplies each input by a weight, adds a bias, and passes the total through an **activation function**. Without an activation, stacked layers collapse into one linear map. `ReLU` is the usual choice: it outputs the input if it is positive, and 0 otherwise. Non-linear activations let networks learn curves.

relu.py

```
import torch
import torch.nn as nn

relu = nn.ReLU()
print(relu(torch.tensor([-2.0, 0.0, 3.5])))
```

Building a network

A neural network is a class that inherits from `nn.Module` (Unit 14). In `__init__` you define the layers. In `forward` you describe how data flows through them. Calling the model, such as `net(x)`, runs `forward`.

network.py

```
import torch
import torch.nn as nn

class TinyNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(4, 8)   # 4 features in, 8 hidden units
        self.act = nn.ReLU()
        self.out = nn.Linear(8, 3)      # 3 classes out

    def forward(self, x):
        x = self.act(self.hidden(x))
        return self.out(x)

net = TinyNet()
batch = torch.rand(5, 4)                # 5 examples, 4 features each
logits = net(batch)
print(logits.shape)                     # torch.Size([5, 3])
```

The output of the last layer, before any softmax, is called the **logits**. Each row is a score for each class. The class with the highest score is the prediction.

Parameters

parameters.py

```
import torch.nn as nn

layer = nn.Linear(4, 8)
print(layer.weight.shape)
print(layer.bias.shape)
print(sum(p.numel() for p in layer.parameters()))
```

Parameters are the weights and biases the network learns. `parameters()` lists them, and `numel()` counts the numbers in each. 4 × 8 weights plus 8 biases is 40 numbers for this layer.

Predict the output THINK FIRST

Shapes and gradients are where most PyTorch errors start.

```
import torch
print(torch.zeros(2, 3).shape)
```

Answer: `torch.Size([2, 3])`

```
import torch
print(torch.arange(4).sum().item())
```

Answer: `6`

```
import torch
x = torch.tensor(2.0, requires_grad=True)
y = x * x
y.backward()
print(x.grad.item())
```

Answer: `4.0`
$d(x^2)/dx = 2x = 4$ at $x = 2$.

```
import torch.nn as nn
print(nn.Linear(4, 8).weight.shape)
```

Answer: `torch.Size([8, 4])`
PyTorch stores the weight as (out_features, in_features).

Practice

▼ **Exercise 1 — Gradient check**

Create a tensor `x` with value 5.0 and `requires_grad=True`. Compute `y = 3 * x ** 2`, call `backward()`, and print `x.grad`. Check that it matches the derivative 6x by hand.

▼ **Exercise 2 — Shape the network**

Change the `TinyNet` above so its hidden layer has 16 units. Predict the new parameter count before you run it, then check. Hint: a Linear layer from *a* to *b* has $a \times b + b$ parameters.

Discussion

1. Why does a network need a non-linear activation? What would happen to three stacked linear layers without one?
2. Gradients tell you which way to move a weight. What would go wrong if the gradient were always zero?
3. Why might a team prefer a smaller network that is easier to explain, even if a bigger one is slightly more accurate?

Key terms

Tensor

A multi-dimensional array in PyTorch that can run on a GPU and track gradients.

dtype

The number type stored in a tensor, such as `float32` or `int64`.

Device

Where a tensor lives: `cpu` or an accelerator such as `cuda`.

Gradient

The rate at which an output changes as a parameter changes. Training moves parameters against the gradient.

Autograd

PyTorch's automatic differentiation: it records operations and computes gradients on `backward()`.

Neuron

A unit that computes a weighted sum of its inputs plus a bias, then applies an activation function.

Activation function

A non-linear function applied after a layer, such as ReLU, so the network can learn curves.

nn.Module

The base class for PyTorch models. Layers are defined in `__init__` and the data flow in `forward`.

Logits

The raw, unnormalized scores a classifier outputs before a softmax or other probability step.

Parameters

The learned numbers of a model, such as weights and biases.

See all terms in the glossary →

UNIT 21 100 min

Training a PyTorch Model

Turn a dataset into tensors, choose a loss and optimizer, run the training loop, evaluate honestly, and save the model.

By the end of this unit you will be able to

- Prepare a dataset as tensors with a train/test split and feature scaling
- Choose a loss function for regression and for classification
- Write the five steps of a training loop: zero gradients, forward, loss, backward, step
- Evaluate a model with `model.eval()` and `torch.no_grad()`
- Save and reload a model with `state_dict`

Note

Where to run this unit. Use Google Colab or a local install with `pip install torch scikit-learn` . PyTorch does not run in the browser sandbox. The code uses a fixed random seed so results repeat closely from run to run; exact decimals can vary slightly between machines. **Run the code blocks in order in one notebook.** Each block uses variables created by the blocks above it.

The training loop

Training is a loop. Each pass over the training data is an **epoch**. For each batch, the model makes predictions, a **loss** measures how wrong they are, and the optimizer nudges the weights to reduce that loss. Everything else in this unit fills in those steps.

1. Clear old gradients: `optimizer.zero_grad()`
2. Forward pass: `logits = model(x)`
3. Compute the loss: `loss = loss_fn(logits, y)`
4. Backward pass: `loss.backward()`
5. Update the weights: `optimizer.step()`

Prepare the data

Start from the iris data used in Unit 19. Split it first, then scale using statistics from the training set only (the same rule as the pipeline in Unit 19). Convert the NumPy arrays to tensors. Classification labels must be `int64` for `CrossEntropyLoss`.

data.py

```
import torch
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.25, random_state=42, stratify=iris.targ
)

scaler = StandardScaler().fit(X_train)          # learn from training data only
X_train = torch.tensor(scaler.transform(X_train), dtype=torch.float32)
X_test = torch.tensor(scaler.transform(X_test), dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.long)
y_test = torch.tensor(y_test, dtype=torch.long)

print(X_train.shape, y_train.dtype)
```

Note

`stratify=iris.target` keeps the same proportion of each species in both sets. It matters when classes are small or uneven.

Model, loss, and optimizer

- **Model:** a small network that maps 4 features to 3 class scores.
- **Loss:** `CrossEntropyLoss` for classification. It takes raw logits and the integer class labels. For regression, `MSELoss` is the usual choice.
- **Optimizer:** `Adam` adjusts each weight using its gradient history. Its learning rate (`lr`) sets the step size. Too large and the loss bounces; too small and training is slow.

setup.py

```
import torch
import torch.nn as nn

torch.manual_seed(0)

model = nn.Sequential(
    nn.Linear(4, 16),
    nn.ReLU(),
    nn.Linear(16, 3)
)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
```

Running the loop

train.py

```
for epoch in range(200):
    model.train()
    optimizer.zero_grad()
    logits = model(X_train)
    loss = loss_fn(logits, y_train)
    loss.backward()
    optimizer.step()

    if (epoch + 1) % 50 == 0:
        print(f"epoch {epoch + 1:3d}  loss {loss.item():.3f}")
```

This loop runs the whole training set through the model at once, which is called full-batch training. Real datasets are too large for that, so they are split into **mini-batches**, usually fetched with a `DataLoader`. The five steps stay the same.

Watch out

Forgetting `optimizer.zero_grad()` makes gradients add up across steps, and training behaves strangely. Put it first in the loop every time.

Evaluate honestly

Switch to evaluation mode with `model.eval()`, which turns off behaviors such as dropout that only apply during training. Wrap the calculation in `torch.no_grad()` so PyTorch does not build the graph needed for gradients. Score on the test set, which the model has never used for training.

evaluate.py

```
model.eval()
with torch.no_grad():
    test_logits = model(X_test)
    predicted = test_logits.argmax(dim=1)
    accuracy = (predicted == y_test).float().mean().item()

print(f"test accuracy: {accuracy:.2f}")
print("above 0.9:", accuracy > 0.9)
```

`argmax(dim=1)` picks the highest score in each row. Comparing with the labels and taking the mean gives the fraction correct.

Save and reload

Save the **state_dict**, which is a dictionary of the learned weights, rather than the whole model object. To reload, build the same architecture first, then load the weights into it.

save_load.py

```
import torch
import torch.nn as nn

model = nn.Sequential(nn.Linear(4, 16), nn.ReLU(), nn.Linear(16, 3))
torch.save(model.state_dict(), "iris_model.pt")

fresh = nn.Sequential(nn.Linear(4, 16), nn.ReLU(), nn.Linear(16, 3))
fresh.load_state_dict(torch.load("iris_model.pt"))
fresh.eval()
print("reloaded")
```

Tip

Newer PyTorch versions may warn about `weights_only` when loading. Passing `weights_only=True` to `torch.load` is the safe choice for a state_dict.

Common problems

Symptom	Likely cause	Fix
Loss is `nan`	Learning rate too high, or bad input values	Lower `lr`; check scaling
Loss does not go down	Learning rate too low, or gradients not flowing	Raise `lr`; check `backward()` and `step()`
Training accuracy high, test accuracy low	Overfitting	More data, smaller model, early stopping
Shape or dtype error	Labels not `long`, or features not `float32`	Cast with `.long()` or `.float()`

Reason before you run

- Without `optimizer.zero_grad()`, what happens to `.grad` between steps? *It accumulates, because PyTorch adds each new backward pass to the old gradients.*

- Why evaluate with `torch.no_grad()`? *No gradients are needed for predictions, so it saves memory and time.*
- Why fit the scaler on the training set only? *So the test set stays unseen, as in Unit 19.*

Practice

▼ Exercise 1 — Fit a line

Train a single `nn.Linear(1, 1)` with `MSELoss` and SGD (`lr=0.05`) to learn the rule `y = 2x + 1` from `x = torch.linspace(0, 1, 20).reshape(-1, 1)`. Train for 500 epochs, then print the learned weight and bias, rounded to one decimal place.

▼ Hint

Build `y = 2 * x + 1` as the target. Then the loop is the same five steps as above.

Discussion

1. Why do we keep a test set that the model never trains on, even when the data is small?
2. The loss went down, but the test accuracy did not improve. List three things you would check, in order.
3. Would you deploy a model that scores 0.95 on a test set you looked at a hundred times while tuning? Why or why not?

Key terms

Epoch

One full pass of the training data through the model.

Batch / mini-batch

A subset of the training data processed together in one step of the loop.

Loss function

A formula that measures how wrong the predictions are. Training minimizes it.

CrossEntropyLoss

The standard loss for classification. It takes raw logits and integer class labels.

Optimizer

The algorithm that updates the weights from their gradients, such as SGD or Adam.

Learning rate

The size of each update step taken by the optimizer.

model.eval()

Switches a model to evaluation behavior, turning off training-only behaviors such as dropout.

torch.no_grad()

A context manager that turns off gradient tracking for faster, lighter evaluation.

state_dict

A dictionary of a model's learned parameters, used to save and load it.

Stratify

Splitting data so each class keeps the same proportion in the training and test sets.

See all terms in the glossary →

UNIT 22 90 min

Capstone and Responsible Data Work

Plan and carry out a complete data project from question to report, and check it for leakage, bias, and privacy before anyone trusts it.

By the end of this unit you will be able to

- Turn a vague question into a measurable prediction task
- Run an end-to-end project: load, clean, explore, model, evaluate, report
- Check a project for leakage, misleading charts, and unfair outcomes
- Protect personal data, including student records
- Write a short model card that states what a model is and is not for

The project

This unit is a project, not a lecture. You will use the skills from Units 1 through 21 on one question you care about. A useful project answers a question someone will act on, uses data you have the right to use, and reports both what the model does well and where it fails.

Step 1: write the question

A question is measurable when you can say what counts as success. "Can we predict which students will need tutoring?" is vague. "Can we flag students whose quarter grade will fall below 70, using only data available by week 4?" is measurable: the target is defined, and the time limit prevents using information from the future.

- **Target:** what exactly are you predicting, and how is it measured?
- **Timing:** what information will be available at prediction time? Anything later is leakage.
- **Action:** what will someone do with the answer? If nothing, the project is an exercise.

Step 2: check the data

project_start.py

```
import io
import pandas as pd

# Replace this text with pd.read_csv("your_file.csv") for real data.
raw = io.StringIO("""student,week1_quiz,week2_quiz,hw_done,final_grade
A01,88,91,5,89
A02,62,58,2,64
A03,75,,4,78
A04,93,95,5,96
A05,50,47,1,55
""")
df = pd.read_csv(raw)
print(df.info())
print(df.isna().sum())
print(df.describe().round(1))
```

Before modeling, look at missing values, impossible numbers (a quiz score of 140), duplicated rows, and the balance of the target. A model trained on a dataset that is 98% one class can score well by ignoring the rare class.

Step 3: avoid leakage

Leakage means a feature contains information you would not have when making the prediction. The most common kind is a column that is recorded after the outcome, such as "final exam taken" when predicting the final grade. Leakage gives scores that look excellent and fail the first time the model meets real data. Remove leaking columns before you split, and keep the split before any fitting, as in Unit 19.

> **Watch out**
> A project that reports 99% accuracy on its first try should be checked for leakage before it is celebrated.

Step 4: model and compare

Start with a simple baseline: always predict the most common class, or predict the average. Any real model must beat the baseline clearly. Then try one or two models, such as logistic regression and a decision tree, using the same split. Report the baseline next to every result.

baseline.py

```
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_breast_cancer

data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.25, random_state=0, stratify=data.targe
)

baseline = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
model = LogisticRegression(max_iter=5000).fit(X_train, y_train)

print("baseline:", round(baseline.score(X_test, y_test), 3))
print("logistic:", round(model.score(X_test, y_test), 3))
```

The breast cancer dataset ships with scikit-learn and is used here only to practice the workflow. It is not a clinical tool.

Step 5: report honestly

A good report has five parts: the question, the data and its limits, the method, the results against the baseline, and the failures. Show at least one chart, labeled as Unit 18 taught. State uncertainty. Say what you did not test.

Step 6: check fairness

A model can be accurate overall and still fail a group. Compute the same metric separately for each group that matters (for example, each class period, gender, or language background). If the results

differ a lot, find out why before deploying. Often the cause is a training set that under-represents the group.

by_group.py

```
import pandas as pd

results = pd.DataFrame({
    "group": ["A", "A", "A", "B", "B", "B"],
    "correct": [1, 1, 0, 1, 0, 0]
})
print(results.groupby("group")["correct"].mean().round(2))
```

Privacy: student and personal data

Student records are sensitive, and in the United States they are protected by law (FERPA). The same caution applies to any personal data. Practical habits:

- Use the minimum data you need. Drop names and IDs before analysis, and keep a separate key only if you must link results back.
- Do not upload identifiable data to a public tool, notebook, or chatbot. Check your school's approved tools first.
- Share aggregated results (counts, averages with enough people in each group), not individual rows.
- Delete working copies you no longer need, and keep the original in its approved location.

> **Tip**
> Before using any dataset, ask: who collected it, for what purpose, and did those people agree to this use? If you cannot answer, do not use it yet.

Step 7: write a model card

A **model card** is a short document that travels with a model. It answers: what the model predicts, what data it was trained on, how it was evaluated, where it works well, where it fails, and what it must not be used for. Writing it forces you to be specific. A template you can copy:

Section	What to write
Purpose	The question, and the decision it supports
Data	Source, date range, size, known gaps
Method	Model type, features, split, random seed
Results	Metric on the held-out test set, compared with the baseline
Groups	Results for each important group, and any gaps
Limits	Where it fails; what it must not be used for
Owner	Who maintains it, and when it will be reviewed

The capstone checklist

1. A measurable question, with target, timing, and action written down
2. Data loaded, inspected, and cleaned; limits noted
3. Leaking columns removed; split done before any fitting
4. Baseline reported next to every model
5. Test set used once, at the end, for the final number
6. At least one labeled chart
7. Group results checked, with gaps explained
8. Personal data removed or protected
9. Model card written

Practice

▼ **Exercise 1 — Find the leak**

A team predicts whether a loan will default. Their features include `days_late_on_loan` and `default_flag`. Which column leaks the answer, and why? Write the one-line fix.

▼ **Hint**

Ask whether you could know this value at the moment of the decision.

Discussion

1. Which is worse in your context: a false alarm (flagging a student who was fine) or a miss (not flagging one who needed help)? How does that choice change the model you should build?
2. Your model is 90% accurate overall, but 60% accurate for one group. What do you do before deploying?
3. Who should sign off on a model that makes decisions about people, and what should they see first?

Key terms

Leakage
When a model is trained on information that would not be available at prediction time, inflating its score.

Baseline
A simple reference model, such as always predicting the most common class, that any real model must beat.

Model card
A short document describing a model's purpose, data, evaluation, limits, and owner.

Fairness (group metrics)
Checking that a model's performance is comparable across the groups it affects.

Unsupervised learning
Learning structure from unlabeled data, such as clustering similar examples together.

FERPA
The US federal law protecting the privacy of student education records.

Aggregation
Reporting summaries (counts, averages) instead of individual records, to protect privacy.

See all terms in the glossary →

UNIT 23 90 min

Working Like a Python Developer

Input from users, generators, match statements, scope keywords, identity versus equality, testing with assert and unittest, virtual environments with pip, and Jupyter or Colab.

By the end of this unit you will be able to

- Read user input with `input()` and convert it safely
- Write a generator with `yield` and explain why it saves memory
- Use a `match` statement to choose among patterns
- Explain `global` and `nonlocal`, and when to avoid them
- Tell `is` from `==`
- Write a test with `assert` and `unittest`
- Create a virtual environment and install packages with pip
- Run notebook code in Jupyter or Google Colab

Note

This is a reference unit. Read it before Units 19 to 21, which use Colab and pip. The sandbox cannot run `input()`, so those examples are shown as code to run on your computer or in Colab.

Reading input

`input(prompt)` pauses the program, shows the prompt, and returns what the user typed as a **string**. Convert it before using it as a number. Anything a user types can be wrong, so pair the conversion with an exception check from Unit 11.

ask_age.py

```
def ask_age():
    while True:
        text = input("Your age: ")
        try:
            age = int(text)
        except ValueError:
            print("Please type a whole number.")
            continue
        if age < 0:
            print("Age cannot be negative.")
            continue
        return age

print("You are", ask_age())
```

Generators: values one at a time

A function that uses `yield` is a **generator**. Each `yield` hands back one value and pauses. The next value is computed only when it is asked for. A list of a million numbers is stored in memory all at once; a generator can produce the same numbers without storing them.

generator.py

```
def countdown(n):
    while n > 0:
        yield n
        n -= 1

print(list(countdown(3)))

total = 0
for value in countdown(4):
    total += value
print(total)
```

Generator expressions from Unit 12 are generators too. Use them when you only need to loop once over a large sequence.

match: choosing by pattern

The `match` statement (Python 3.10 and later) compares a value against several patterns. It is clearer than a long `elif` chain when you are checking one value against fixed options. The `_` pattern is the default.

match.py

```
def describe(code):
    match code:
        case 200:
            return "OK"
        case 404:
            return "Not found"
        case 500 | 503:
            return "Server problem"
        case _:
            return "Unknown"

print(describe(404))
print(describe(503))
print(describe(302))
```

global and nonlocal

Unit 6 showed that assigning a name inside a function creates a local variable. `global name` tells Python to assign the module-level variable instead. `nonlocal name` does the same for a variable in an enclosing function.

global_nonlocal.py

```
count = 0

def bump():
    global count
    count += 1

def make_counter():
    total = 0
    def add():
        nonlocal total
        total += 1
        return total
    return add

bump()
bump()
print(count)

counter = make_counter()
counter()
print(counter())
```

Watch out

Use `global` sparingly. A function that changes a global is hard to test and hard to reason about. Passing values in and returning results (Unit 6) is almost always clearer.

is versus ==

`==` asks whether two values are equal. `is` asks whether they are the **same object** in memory. Use `is` for `None` and for the singletons `True` and `False`; use `==` for everything else.

identity.py

```
a = [1, 2]
b = [1, 2]
c = a
print(a == b, a is b)
print(a is c)
print(None is None)
```

Testing with assert

`assert condition, message` stops the program with an `AssertionError` when the condition is False. It is a quick check during development. Tests are written the same way, but kept in files and run automatically.

assert_demo.py

```
def add_tax(price, rate=0.07):
    return round(price * (1 + rate), 2)

assert add_tax(100) == 107.0, "100 with tax should be 107.0"
assert add_tax(0) == 0, "zero stays zero"
print("all checks passed")
```

Testing with unittest

The `unittest` module (in the standard library) organizes tests into classes. Each method whose name starts with `test_` is a test. Run the file with `python test_tax.py`, and you see how many tests passed.

test_tax.py

```
import unittest

def add_tax(price, rate=0.07):
    return round(price * (1 + rate), 2)

class TestAddTax(unittest.TestCase):
    def test_basic(self):
        self.assertEqual(add_tax(100), 107.0)

    def test_zero(self):
        self.assertEqual(add_tax(0), 0)

if __name__ == "__main__":
    unittest.main()
```

> **Tip**
> Write the test before you fix a bug. If the test fails, you have reproduced the bug. When it passes, the bug stays fixed even after future changes.

Virtual environments and pip

A **virtual environment** is a private folder of packages for one project, so different projects can use different versions. Create one once per project, activate it, then install with `pip`. These commands are typed in a terminal, not in Python.

terminal (not Python)

```
python -m venv .venv
# Windows:   .venv\Scripts\activate
# macOS/Linux: source .venv/bin/activate
pip install numpy pandas scikit-learn matplotlib
pip freeze > requirements.txt
```

`pip freeze` records the exact versions you installed in `requirements.txt`. Someone else can recreate the environment with `pip install -r requirements.txt`. Keeping this file is the

difference between code that runs once and code that runs again.

Jupyter and Google Colab

A **notebook** mixes code cells with text, and each cell runs in the same session. It is ideal for exploring data and for lessons like these. **Google Colab** runs notebooks in your browser with NumPy, pandas, scikit-learn, matplotlib, and PyTorch already installed (PyTorch is used in Units 20 and 21). To install something extra in Colab, run `!pip install package-name` in a cell.

- Run a cell with Shift+Enter. The variables persist between cells.
- If results look wrong, use Runtime → Restart session and run everything from the top. Hidden state from earlier cells is a common source of confusing results.
- Notebooks are for exploring. Move code you rely on into a `.py` file with functions and tests.

Debugging with print and the debugger

The fastest debugger is a `print()` that shows the value you are unsure about, with a label. Remove it when you are done. For longer problems, the built-in debugger `breakpoint()` pauses the program at that line and lets you inspect variables. Type `c` to continue and `q` to quit.

Predict the output THINK FIRST

Reason through the behavior, then check it.

```
def g():
    yield 1
    yield 2
print(list(g()))
```

Answer: `[1, 2]`

```
x = None
print(x is None)
```

Answer: `True`

```
print([1] == [1], [1] is [1])
```

Answer: `True False`
Two separate list objects with the same contents.

```
def f(n):
    match n:
        case 0:
            return "zero"
        case _:
            return "other"
print(f(5))
```

Answer: `other`

```
count = 1
def f():
    global count
    count = 5
f()
print(count)
```

Answer: `5`

Practice

▼ **Exercise 1 — Even numbers generator**

Write a generator `evens(limit)` that yields the even numbers from 0 up to but not including `limit`. Print `list(evens(10))`.

▼ **Exercise 2 — Test a function**

Write a function `is_leap(year)` using the rule: divisible by 4, except centuries that are not divisible by 400. Write two `assert` statements: 2024 is leap, and 1900 is not.

Discussion

1. Why does a generator matter for a file with ten million lines? What would a list of those lines cost?
2. You inherit a project with no `requirements.txt`. What would you do first, and why?
3. Which tool in this unit would have saved you the most time in a project you already finished?

Key terms

input()

A built-in that shows a prompt and returns the user's typed text as a string.

Generator

A function that uses `yield` to produce values one at a time, pausing between them.

yield

A statement that returns a value from a generator and pauses until the next value is requested.

match statement

A statement that compares a value against patterns and runs the first pattern that fits (Python 3.10+).

global / nonlocal

Keywords that let a function assign a variable from the module level (`global`) or an enclosing function (`nonlocal`).

is

The identity operator. True when two names refer to the same object in memory.

assert

A statement that raises `AssertionError` when its condition is False, used for quick checks and tests.

unittest

The standard-library testing framework. Tests are methods named `test_...` inside a `TestCase` class.

Virtual environment

An isolated folder of installed packages for one project, created with `python -m venv`.

pip

The package installer for Python. Installs libraries from the Python Package Index.

Notebook

A document of code and text cells, each runnable in a shared session. Jupyter and Google Colab both use notebooks.

See all terms in the glossary →